

Computación Científica

Parcial 1

Factorizaciones LU, PLU y QR

Interpolación polinómica mediante la matriz de Vandermonde

Maria Lucía Castillo García

Juliana González Sánchez

Miguel Ángel Padilla Rosero

Docente: Juan Fernando Paz

14 de septiembre de 2026

Índice

1. Componente teórico	2
1.1. Relación entre la invertibilidad de A y la existencia de una solución única para $Ax = b$	2
1.2. Procedimiento matemático en que se fundamentan LU y QR	2
1.3. Propiedades de interés y ganancia para las soluciones numéricas	3
1.4. Condiciones de aplicabilidad y modificaciones necesarias	5
1.5. Ventajas en computación científica	6
1.6. Aplicabilidad de cada método a las matrices dadas	7
2. Componente práctico	9
2.1. Diagnóstico previo	9
2.2. Errores numéricos y tiempos de ejecución	10
2.3. Sobre $P(1)$ y $Q(1)$	11
2.4. Visualización y conclusiones	11
3. Código y reproducibilidad	13
4. Contribuciones individuales	13
5. Declaración del uso de inteligencia artificial	15
5.1. Herramienta y propósito	15
5.2. Prompts utilizados	15
5.3. Elementos aceptados, modificados o rechazados	15
5.4. Verificación y aprendizajes	16
A. Anexo A. Capturas de los prompts utilizados	17

1. Componente teórico

1.1. Relación entre la invertibilidad de A y la existencia de una solución única para $Ax = b$

Caso general

Si A es una matriz cuadrada e invertible, entonces existe A^{-1} , lo que garantiza que el sistema $Ax = b$ tiene solución única $x = A^{-1}b$ para cualquier b . Esto es una consecuencia directa de que el determinante sea distinto de cero, lo cual equivale a que las columnas de A sean linealmente independientes. Si A no fuera invertible, el sistema no tendría solución única: podría no tener solución (incompatible) o tener infinitas soluciones (compatible indeterminado). Conviene precisar que la invertibilidad solo está definida para matrices cuadradas.

LU

Esta relación entre la invertibilidad y la solución única de un sistema es la que hace posible aplicar la factorización LU, donde el objetivo es hallar la solución única de forma eficiente reescribiendo $A = LU$ y resolviendo dos sistemas triangulares, $Ly = b$ y $Ux = y$.

Además, en la factorización de Crout la invertibilidad de A se relaciona con que los elementos de la diagonal de L , que actúan como pivotes durante el procedimiento, sean diferentes de cero. Esto permite realizar las divisiones necesarias para calcular L y U sin inconvenientes.

PLU

Aunque el teorema asegura que si A es invertible la solución única existe, en la práctica la eliminación de Gauss directa o la factorización LU pura falla si aparece un pivote cero. Sin embargo, precisamente porque A es invertible, se garantiza que siempre existe al menos un elemento no nulo en las filas restantes de esa columna. En consecuencia, siempre es posible encontrar una matriz de permutación P que reordene las filas para completar la eliminación, obteniendo

$$PA = LU \quad (\text{o equivalentemente } A = P^T LU),$$

donde P es una matriz de permutación, L es triangular inferior unitaria y U es triangular superior con elementos diagonales no nulos ($u_{ii} \neq 0$). De este modo, la invertibilidad de A asegura que el sistema permutado $PAx = Pb$ se resuelva de forma única y estable en dos pasos triangulares:

$$\begin{aligned} Ly &= Pb && (\text{sustitución hacia adelante para hallar } y) \\ Ux &= y && (\text{sustitución hacia atrás para hallar } x) \end{aligned}$$

recuperando de forma exacta la solución única

$$x = U^{-1}L^{-1}Pb = A^{-1}b.$$

QR

Si A es invertible, entonces sus columnas son linealmente independientes y el sistema $Ax = b$ tiene una única solución para cualquier vector b . Al factorizar $A = QR$, el sistema puede escribirse como $Rx = Q^T b$, ya que Q es ortogonal. Además, la independencia lineal de las columnas de A implica que los elementos diagonales de R son distintos de cero, por lo que R es invertible. En consecuencia, el sistema triangular puede resolverse mediante sustitución hacia atrás y se obtiene una única solución.

1.2. Procedimiento matemático en que se fundamentan LU y QR

LU

Se fundamenta en la eliminación de Gauss. Durante este procedimiento se utilizan diversos multiplicadores para convertir la matriz A en una matriz triangular superior. En la factorización LU, en vez de descartar estos multiplicadores, se organizan en la matriz triangular inferior L , mientras que la matriz obtenida al finalizar la eliminación queda como la matriz triangular superior U . Entonces, la matriz original puede expresarse como

$$A = LU,$$

y esta factorización permite resolver $Ax = b$ en dos pasos: $Ly = b$ mediante sustitución hacia adelante, y luego $Ux = y$ mediante sustitución hacia atrás.

QR

Se basa en el proceso de ortogonalización de Gram–Schmidt. Durante este procedimiento se toman las columnas de A una por una y, a cada una, se le restan sus proyecciones sobre las direcciones ya construidas, calculadas mediante productos punto. Lo que queda después de esa resta es un vector que ya no tiene ninguna componente en las direcciones anteriores, es decir, es ortogonal a todas ellas. Luego, ese residuo se divide entre su norma para volverlo unitario. Repitiendo el proceso columna por columna se obtiene un conjunto de vectores ortonormales, que son las columnas de la matriz Q .

En la factorización QR, al igual que ocurre en LU con los multiplicadores, los coeficientes obtenidos de las proyecciones y las normas utilizadas durante la normalización quedan registrados en la matriz triangular superior R . En particular, cada columna a_j de A puede expresarse como una combinación de los vectores ortonormales construidos hasta ese momento:

$$a_j = r_{1j}q_1 + r_{2j}q_2 + \cdots + r_{jj}q_j.$$

Así, la matriz original puede expresarse como

$$A = QR,$$

y esta factorización permite resolver $Ax = b$ en dos pasos: primero se calcula $Q^T b$, aprovechando que $Q^{-1} = Q^T$, y luego se resuelve $Rx = Q^T b$ mediante sustitución hacia atrás.

1.3. Propiedades de interés y ganancia para las soluciones numéricas

LU

Propiedades de interés.

- L es triangular inferior y U es triangular superior: el método descompone la matriz original A en otras dos.
- En el método de Crout, U tiene unos en su diagonal, mientras que L tiene los valores reales (no necesariamente 1) en la suya.
- **Propiedad estructural clave:** una matriz triangular se resuelve por sustitución en lugar de eliminación, porque cada ecuación introduce una sola incógnita nueva a la vez (hacia adelante en L , hacia atrás en U).

Ganancia en soluciones numéricas.

- **Fácil resolución:** como L y U son triangulares, pueden resolverse mediante sustitución hacia adelante y hacia atrás, sin necesidad de realizar nuevamente la eliminación de Gauss.
- **Reutilización de la factorización:** si se deben resolver varios sistemas con la misma matriz A pero distintos vectores b , la factorización $A = LU$ se calcula una única vez, con un costo aproximado de $O(n^3)$, similar al de la eliminación de Gauss. La ganancia viene después, pues cada sistema adicional solo requiere resolver $Ly = b$ y $Ux = y$ por sustitución, con un costo aproximado de $O(n^2)$ cada uno.

PLU

Propiedades de interés.

- P (matriz de permutación) se forma reordenando las filas de la matriz identidad. Es una matriz ortogonal discreta cuya inversa es simplemente su transpuesta, $P^{-1} = P^T$, y su determinante es $\det(P) = \pm 1$.
- **Acotamiento de multiplicadores:** al aplicar pivoteo parcial, seleccionando en cada etapa el elemento de mayor magnitud de la columna, los multiplicadores de L quedan estrictamente acotados: $|l_{ij}| \leq 1$.
- L y U conservan su estructura triangular.

Ganancia en soluciones numéricas.

- **Estabilidad numérica y control de errores:** al garantizar que ningún multiplicador supere 1, se evita el crecimiento descontrolado de los números intermedios y se minimiza la propagación de errores de redondeo.
- **Prevención de división por cero:** permite resolver sistemas donde aparecen ceros en la diagonal durante la eliminación gaussiana sin que el algoritmo se detenga.
- **Costo computacional casi nulo para P :** multiplicar por P o aplicar P^T no requiere operaciones de punto flotante; computacionalmente solo reordena un vector de índices en memoria.
- Mantiene la ventaja de resolución rápida de LU en dos etapas triangulares con costo $O(n^2)$.

QR

Propiedades de interés.

- **Q tiene columnas ortonormales:** sus columnas son perpendiculares entre sí y todas de longitud 1, lo que se resume en $Q^T Q = I$. La consecuencia inmediata es que su inversa no hay que calcularla: $Q^{-1} = Q^T$, es decir, basta con transponer.
- **Q conserva las longitudes:** multiplicar un vector por Q no cambia su tamaño, $\|Qx\| = \|x\|$. Q únicamente reorienta, nunca estira ni encoge. De esta propiedad se desprende casi toda la ventaja numérica de la factorización.
- **R es triangular superior:** al igual que U en LU, permite resolver por sustitución.
- **Propiedad estructural clave:** la ortogonalidad reemplaza el cálculo de una inversa por una transposición, y la forma triangular reemplaza la eliminación por sustitución. Entre las dos, resolver el sistema no exige invertir ninguna matriz.

Ganancia en soluciones numéricas.

- **Estabilidad numérica:** como Q no modifica las longitudes, los errores de redondeo tampoco se amplifican al pasar por ella. Mientras que el pivoteo parcial en PLU *controla* el crecimiento de los valores intermedios acotando los multiplicadores, en QR ese crecimiento sencillamente no llega a producirse. Por eso QR resulta preferible cuando la matriz está mal condicionada.
- **No requiere pivoteo:** la factorización QR existe para cualquier matriz sin necesidad de reordenar filas, ya que el procedimiento nunca divide entre elementos de A , sino únicamente entre normas de vectores.
- **Reutilización de la factorización:** igual que en LU, se calcula una sola vez y cada sistema adicional cuesta aproximadamente $O(n^2)$: una multiplicación por Q^T y una sustitución hacia atrás.

- **Información sobre la propia matriz:** la diagonal de R indica de inmediato si A es invertible y qué tan cerca está de no serlo, pues valores diagonales muy pequeños advierten que las columnas son casi dependientes.
- **El costo:** QR requiere aproximadamente el doble de operaciones que LU. Es el precio de la estabilidad, y conviene pagarlo cuando la matriz es delicada o cuando el sistema no es cuadrado.

1.4. Condiciones de aplicabilidad y modificaciones necesarias

LU

- Para que el método de Crout pueda realizarse sin modificaciones, los valores de la diagonal de L que se van obteniendo en cada paso deben ser diferentes de cero, ya que son los divisores necesarios para calcular las entradas de U . Podemos expresar esta condición de tal forma que los valores de la diagonal principal sean distintos de cero.
- Que A sea invertible significa que $\det(A) \neq 0$, por lo que el sistema $Ax = b$ tiene solución única. Sin embargo, esto no garantiza que el método de Crout pueda realizarse directamente, ya que durante el procedimiento pueden aparecer ceros en los elementos de la diagonal de L que se utilizan como divisores. Por ejemplo, para

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix},$$

la matriz es invertible pues $\det(A) \neq 0$, pero en el primer paso de Crout tendríamos $l_{11} = a_{11} = 0$, por lo que no es posible continuar sin un cambio de filas (aquí es donde entra PLU con pivoteo, realmente).

- ¿Cuándo realizar cambios? Es necesario realizar un cambio cuando, durante el proceso de Crout, alguno de los elementos de la diagonal de L que se necesita como divisor es igual a cero. En ese caso el algoritmo no puede continuar directamente, pues implicaría una división entre cero. Esto puede ocurrir incluso cuando A es invertible, por lo que no significa necesariamente que el sistema no tenga solución única, sino que el procedimiento de Crout no puede continuar con ese orden de filas.

PLU

Condiciones sobre la matriz.

- Se puede aplicar a cualquier matriz cuadrada invertible ($A \in \mathbb{R}^{n \times n}$ con $\det(A) \neq 0$).
- A diferencia de la factorización LU pura, que exige que todos los menores principales dominantes sean distintos de cero o que la matriz sea diagonalmente dominante, PLU no impone restricciones sobre la diagonal original ni sobre los menores principales. Siempre existe para cualquier matriz no singular.

¿Cuándo es necesaria una modificación y en qué consiste? La modificación sobre el sistema original consiste en multiplicar la matriz A por la izquierda por una matriz de permutación P , transformando A en PA y, por coherencia matemática, permutar también el vector de términos independientes (b pasa a ser Pb), de modo que el sistema equivalente sea $PAx = Pb$.

¿Cuándo es indispensable realizarla?

1. **Por imposibilidad algebraica (pivote cero):** cuando en alguna etapa k de la eliminación aparece un cero en la diagonal ($a_{kk} = 0$). Sin permutar filas, el cálculo del multiplicador $l_{ik} = a_{ik}/a_{kk}$ generaría una división por cero inmediata.

2. **Por estabilidad numérica (pivoteo parcial):** incluso si el pivote no es exactamente cero, si tiene una magnitud muy pequeña en comparación con los demás elementos de su columna, dividir por él genera multiplicadores gigantescos que amplifican de forma catastrófica los errores de redondeo.

Procedimiento de la modificación. En cada etapa k se busca el elemento de mayor valor absoluto en la columna k desde la fila k hacia abajo, y se intercambia la fila del pivote con dicha fila mediante P , asegurando que todos los multiplicadores queden acotados: $|l_{ik}| \leq 1$.

QR

Condiciones sobre la matriz.

- Puede aplicarse directamente siempre que ninguna columna de A sea combinación de las demás, es decir, que todas aporten una dirección distinta (lo que se conoce como tener rango completo por columnas). Cuando A es cuadrada, esto equivale simplemente a que A sea invertible.
- A diferencia de LU y PLU, no exige que A sea cuadrada: basta con que tenga al menos tantas filas como columnas. Es la única de las tres factorizaciones que puede aplicarse a sistemas con más ecuaciones que incógnitas.
- Tampoco impone ninguna condición sobre la diagonal de A ni sobre sus menores principales. La razón es que el procedimiento nunca divide entre elementos de A : las únicas divisiones que hace son entre las normas de los vectores que va obteniendo al restar las proyecciones, y una norma solo vale cero si el vector es nulo, lo cual ocurre únicamente cuando esa columna no aportaba ninguna dirección nueva.
- Por eso en QR no puede aparecer un pivote cero por culpa del orden de las filas, y nunca hace falta permutar para que la factorización exista. La misma matriz $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ que impide continuar el método de Crout sin intercambio de filas se factoriza en QR sin ninguna dificultad.

¿Cuándo es necesaria una modificación y en qué consiste? Cabe aclarar que por modificación entendemos un cambio sobre la matriz original del sistema, como la permutación de filas que introduce PLU, y no un cambio en el algoritmo empleado.

Bajo esa premisa, QR no requiere ninguna modificación. Mientras que el método de Crout puede detenerse ante una matriz perfectamente invertible por el solo hecho de cómo están ordenadas sus filas, lo que obliga a introducir P , en QR ese problema no existe: el procedimiento nunca divide entre elementos de A , sino entre las normas de los residuos que va obteniendo, de modo que ningún orden de filas ni de columnas puede provocar una división entre cero.

El único caso en que Gram-Schmidt no puede llevarse a cabo es cuando alguna columna de A es combinación de las otras. Pero conviene notar que ese obstáculo es de naturaleza distinta al de LU: allí el problema estaba en el orden y se resolvía reordenando; aquí el problema está en la matriz misma. La dependencia lineal entre columnas no depende del orden en que se escriban, de modo que ninguna permutación de filas ni de columnas la elimina.

1.5. Ventajas en computación científica

LU

- **Mayor eficiencia al resolver varios sistemas:** si se tiene la misma matriz A y diferentes vectores b , la factorización $A = LU$ se realiza una sola vez. Luego cada nuevo sistema se resuelve mediante sustituciones, evitando repetir la eliminación de Gauss, que requiere aproximadamente $O(n^3)$ operaciones.

- **Menor costo para cada sistema adicional:** una vez obtenida la factorización, resolver cada nuevo sistema requiere aproximadamente $O(n^2)$ operaciones. Esto representa una ventaja importante en problemas grandes o en simulaciones donde se resuelven muchos sistemas.
- **Amplia aplicación práctica:** es una de las factorizaciones utilizadas en métodos y programas de cálculo científico para resolver sistemas lineales presentes en problemas de ingeniería, física, economía y otras áreas.

PLU

- **Robustez general sin pérdida de velocidad:** mantiene la misma velocidad asintótica que LU, aproximadamente $\frac{2}{3}n^3$ operaciones, pero garantiza que el algoritmo no colapsará por divisiones por cero ante ninguna matriz invertible.
- **Estabilidad numérica garantizada:** al implementar pivoteo parcial se asegura que todos los multiplicadores cumplan $|l_{ij}| \leq 1$. Esto frena el crecimiento artificial de los números intermedios y previene la amplificación descontrolada de errores de redondeo en aritmética finita.
- **Estándar en librerías científicas:** es el resolvidor directo universal implementado en paquetes de cómputo de alto rendimiento como LAPACK (rutina `dgesv`), SciPy (`lu_factor` / `lu_solve`) y MATLAB (el operador `\`).
- **Costo de permutación casi nulo:** las permutaciones de filas se realizan a nivel de índices en memoria, sin consumir operaciones de punto flotante.

QR

- **Estabilidad numérica sin necesidad de pivoteo:** como las columnas de Q son ortogonales, multiplicar por ella no altera el tamaño de los vectores y, por tanto, no amplifica los errores de redondeo. Mientras PLU debe recurrir al pivoteo para frenar el crecimiento de los valores intermedios, en QR ese crecimiento no llega a producirse.
- **Aplicable a sistemas con más ecuaciones que incógnitas:** LU y PLU no funcionan con matrices rectangulares, mientras que QR sí, entregando la mejor solución aproximada o solución de mínimos cuadrados. Es la herramienta habitual para ajustar modelos a datos experimentales.
- **Reutilización de la factorización:** igual que en LU, se calcula una sola vez y cada sistema adicional cuesta aproximadamente $O(n^2)$, pues solo requiere una multiplicación por Q^T y una sustitución hacia atrás.
- **Costo mayor que LU:** requiere aproximadamente el doble de operaciones, por lo que para sistemas cuadrados bien condicionados suele preferirse LU. La ventaja de QR se aprovecha cuando la estabilidad es crítica o cuando el sistema no es cuadrado.

1.6. Aplicabilidad de cada método a las matrices dadas

Se consideran las matrices

$$A_1 = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix} \quad A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} \quad A_3 = \begin{pmatrix} 0 & 1 \\ 1 & 2 \end{pmatrix} \quad A_4 = \begin{pmatrix} 1 & 1 \\ 1 & 1 + 10^{-12} \end{pmatrix}$$

Cuadro 1: Aplicabilidad de cada método

Matriz	LU	PLU	QR
A_1	SÍ	SÍ	SÍ
A_2	NO	NO	NO
A_3	NO	SÍ	SÍ
A_4	SÍ*	SÍ*	SÍ*

(*) **Mal condicionada:** la matriz es invertible, pero tan próxima a ser singular que errores diminutos en los datos o en el redondeo producen cambios grandes en la solución. Los tres métodos pueden aplicarse; lo que advierte la marca es que la solución calculada pierde precisión, en mayor o menor medida según el método.

Explicación breve del porqué

Cuadro 2: Explicación breve del porqué

Matriz	LU	PLU	QR
A_1 $\det = 2 \neq 0$, invertible.	El primer pivote es $2 \neq 0$, así que la eliminación procede sin modificaciones.	También aplica; el pivoteo intercambia las filas 1 y 2 porque $ 4 > 2 $.	Sus columnas son linealmente independientes, de modo que Gram–Schmidt se completa sin dividir entre cero.
A_2 $\det = 0$, singular; la segunda columna es el doble de la primera.	El segundo pivote queda en cero, por lo que no hay solución única.	Ningún intercambio de filas evita ese cero, pues el problema es la dependencia entre columnas y no el orden de las filas.	Al ser las columnas dependientes, el residuo de la segunda es cero y no puede normalizarse. Aunque la factorización sigue existiendo, R no resulta invertible ($r_{22} = 0$).
A_3 $\det = -1 \neq 0$, invertible, pero con un cero en la diagonal.	El primer pivote es cero, lo que provoca una división entre cero.	El pivoteo intercambia las filas 1 y 2 y la eliminación se completa sin problema.	Opera sobre las columnas y no sobre la diagonal, así que el cero en a_{11} no lo afecta.
A_4 $\det = 10^{-12} \neq 0$, invertible pero mal condicionada.	Aplica, pero la sustitución hacia atrás divide entre 10^{-12} , lo que amplifica los errores de redondeo.	Aplica sin necesidad de intercambios, aunque la precisión sigue degradada, ya que el mal condicionamiento es propio de la matriz y el pivoteo no lo corrige.	Aplica, y es el más adecuado aquí: al ser Q ortogonal, no altera el tamaño de los vectores y por tanto no amplifica los errores.

2. Componente práctico

2.1. Diagnóstico previo

Antes de realizar el ejercicio se plantea el siguiente diagnóstico, donde se detalla qué suponemos que ocurrirá en cada caso, basándonos en las respuestas del componente teórico.

Tanto para LU como para QR, el sistema que se debe resolver es $Va = y$, donde V es la matriz de Vandermonde formada a partir de los valores x_i y sus diferentes potencias. En el primer caso tenemos 20 puntos, con valores de x entre 0 y 38, y un polinomio de grado 19. En el segundo caso tenemos 200 puntos, con valores de x entre 0 y 398, y un polinomio de grado 199.

LU

Al construir la matriz de Vandermonde, las primeras columnas tienen valores pequeños, mientras que las columnas correspondientes a potencias más altas pueden tener valores muy grandes. Esto hace que las columnas tengan escalas muy diferentes y que la matriz esté mal condicionada.

Aunque la matriz es invertible porque todos los x_i son diferentes, esperamos que el caso de 200 puntos esté mucho peor condicionado que el de 20 puntos, debido a que se utilizan potencias mucho más altas. Por esto, esperamos que los cálculos con LU sean menos precisos en el segundo caso y que los errores al obtener los coeficientes y evaluar el polinomio sean mayores.

QR

Al igual que con LU, el sistema que debemos resolver es $Va = y$. En los dos casos la matriz está mal condicionada debido a que sus columnas pueden tener valores de tamaños muy diferentes, especialmente por las potencias altas de los x_i . Este problema será aún mayor en el caso de 200 puntos, donde se llega hasta potencias de grado 199.

A pesar de esto, esperamos que QR sea más estable numéricamente que LU, ya que utiliza matrices ortogonales, que ayudan a evitar la amplificación de los errores durante los cálculos. Por esta razón esperamos obtener resultados más precisos con QR, especialmente al compararlo con LU en el caso de grado 19.

Para el polinomio de grado 19 esperamos que QR produzca un error relativamente pequeño y posiblemente menor que el obtenido con LU, aunque la matriz ya presenta cierto mal condicionamiento. Para el de grado 199 esperamos que los errores sean mayores debido al fuerte mal condicionamiento, aunque QR debería comportarse de manera más estable.

En cuanto al tiempo de ejecución, esperamos que LU y QR tengan tiempos del mismo orden, ya que ambos tienen un costo computacional aproximado de $O(n^3)$. Aunque, creemos igualmente que QR puede llegarse a demorar más.

A continuación se presenta la salida del diagnóstico ejecutado por el programa:

Listing 1: Salida del diagnóstico previo (opción 1 del menú)

```
DIAGNÓSTICO PREVIO
=====

--- P (20 puntos) ---
Mayor valor de x : 38
Mayor potencia necesaria : 38^19 =~ 10^30.0
Límite de float64 : 10^308
Matriz de Vandermonde finita: True
Número de condicion de V : 9.441e+30
Datos y[i] correctos : 5 de 20
Primer dato corrupto : indice 5, x = 10

--- Q (200 puntos) ---
Mayor valor de x : 398
Mayor potencia necesaria : 398^199 =~ 10^517.4
Límite de float64 : 10^308
```

```

Matriz de Vandermonde finita: False
Entradas infinitas en V : 11188 de 40000
Datos y[i] correctos : 1 de 200
Primer dato corrupto : indice 1, x = 2

```

Lectura del diagnóstico:

- El polinomio real es $p(t) = 1 + t + t^2 + \dots$, por eso $p(1)$ vale 20 y 200.
- Los $y[i]$ se entregaron en int64 y se desbordaron: la mayoría ya viene mal.
- La matriz de Vandermonde esta pesimamente condicionada, y en el caso de 200 puntos ni siquiera cabe en float64.
- Por tanto se espera que ningún método recupere los coeficientes correctos.

2.2. Errores numéricos y tiempos de ejecución

Cuadro 3: Resultados por método

Polinomio	Método	Tiempo (s)	Residuo $\ Va - y\ $	Valor en $x = 1$	Error relativo
P (grado 19)	LU	0,002237	$4,86 \times 10^{17}$	$-4,87 \times 10^{21}$	$2,43 \times 10^{20}$
P (grado 19)	QR	0,003406	$5,57 \times 10^{19}$	$-3,65 \times 10^{19}$	$1,83 \times 10^{18}$
Q (grado 199)	LU	N/A	N/A	N/A	N/A
Q (grado 199)	QR	N/A	N/A	N/A	N/A

Cuadro 4: Coeficientes obtenidos. Los correctos son todos iguales a 1

	a_0	a_1	a_2	a_3	a_4	a_5
Exactos	1	1	1	1	1	1
LU	1,00	$-3,55 \times 10^{22}$	$6,08 \times 10^{22}$	$-4,50 \times 10^{22}$	$1,95 \times 10^{22}$	$-5,59 \times 10^{21}$
QR	$5,03 \times 10^{18}$	$-9,73 \times 10^{19}$	$7,37 \times 10^{19}$	$-2,07 \times 10^{19}$	$2,92 \times 10^{18}$	$-2,36 \times 10^{17}$

Listing 2: Tabla comparativa generada por el programa (opción 4 del menú)

```

TABLA COMPARATIVA
=====

```

```

Pol. Metodo Tiempo (s) Residuo Valor en 1 Error rel.
-----

```

```

P LU 0.001325 4.864e+17 -4.8659e+21 2.433e+20
P QR 0.003554 5.567e+19 -3.6530e+19 1.827e+18
Q LU No aplica No aplica No aplica No aplica
Q QR No aplica No aplica No aplica No aplica

```

En donde evidentemente nos podemos percatar que:

- **La diagonal de R confirma lo anticipado en el componente teórico.** Sus valores se extienden a lo largo de veintiocho órdenes de magnitud, desde 4,47 hasta $1,12 \times 10^{29}$. Como cada elemento de esa diagonal mide cuánta dirección nueva aporta la columna correspondiente, una dispersión así indica que las columnas de la matriz de Vandermonde son casi linealmente dependientes entre sí.
- **Q perdió su ortogonalidad.** La medición $\|Q^T Q - I\| = 10,46$ debería haber dado un valor cercano a cero. Este es precisamente el comportamiento que la teoría advierte para el proceso de Gram-Schmidt clásico cuando las columnas son casi dependientes: las restas sucesivas de proyecciones acumulan errores de redondeo y las columnas de Q dejan de ser perpendiculares entre sí. Al perderse esa propiedad, el paso $Rx = Q^T y$ deja de ser válido.
- **Ser el mejor método no bastó.** Ninguna de las dos factorizaciones puede compensar una matriz con condicionamiento del orden de 10^{31} ni unos datos de entrada ya corrompidos.

- **El caso de grado 199 no llegó a plantearse**, pues las potencias requeridas (hasta $398^{199} \approx 10^{517}$) exceden el rango de `float64`, que llega a 10^{308} . Un 28 % de las entradas de la matriz quedaron en infinito.
- **Sobre el tiempo**, se observa que QR tardó aproximadamente una vez y media lo que tardó LU. Esto es coherente con que su costo teórico sea mayor, aunque la diferencia concreta depende del equipo en que se ejecute.

2.3. Sobre $P(1)$ y $Q(1)$

Cuadro 5: Valores obtenidos en $x = 1$ y sus errores

	Valor esperado	Valor obtenido		Error relativo	
		LU	QR	LU	QR
$P(1)$	20	$-4,87 \times 10^{21}$	$-3,65 \times 10^{19}$	$2,43 \times 10^{20}$	$1,83 \times 10^{18}$
$Q(1)$	200	—	—	—	—

Suponemos que estos errores se deben a cuatro causas:

1. **Los datos de entrada ya venían equivocados.** Los valores y_i se entregaron en `int64` y se desbordaron: de los 20 puntos de P solo 5 son correctos, y de los 200 de Q solo 1. Aun resolviendo con precisión perfecta, el polinomio obtenido pasaría por puntos que no son los verdaderos.
2. **La matriz de Vandermonde está pésimamente condicionada.** Sus columnas son potencias sucesivas de los mismos números y se vuelven casi indistinguibles entre sí, lo que la acerca muchísimo a ser singular. Con un condicionamiento del orden de 10^{31} y una aritmética de unas 16 cifras, la solución calculada no conserva ninguna cifra válida.
3. **El valor $x = 1$ es especialmente revelador.** Como $p(1)$ es la suma de todos los coeficientes, cualquier error en ellos se acumula en ese único punto en lugar de compensarse. Los coeficientes obtenidos alternan entre magnitudes de $\pm 10^{22}$, de modo que su suma queda en el orden de 10^{21} en vez de 20.
4. **En el caso de Q el error no es medible**, porque el sistema nunca pudo plantearse: las potencias hasta 398^{199} desbordan el punto flotante antes de cualquier factorización.

2.4. Visualización y conclusiones

Se presentan las gráficas de los dos polinomios de grado 19, obtenidos con LU y con QR. Los polinomios de grado 199 no pudieron graficarse porque su sistema no llegó a plantearse.

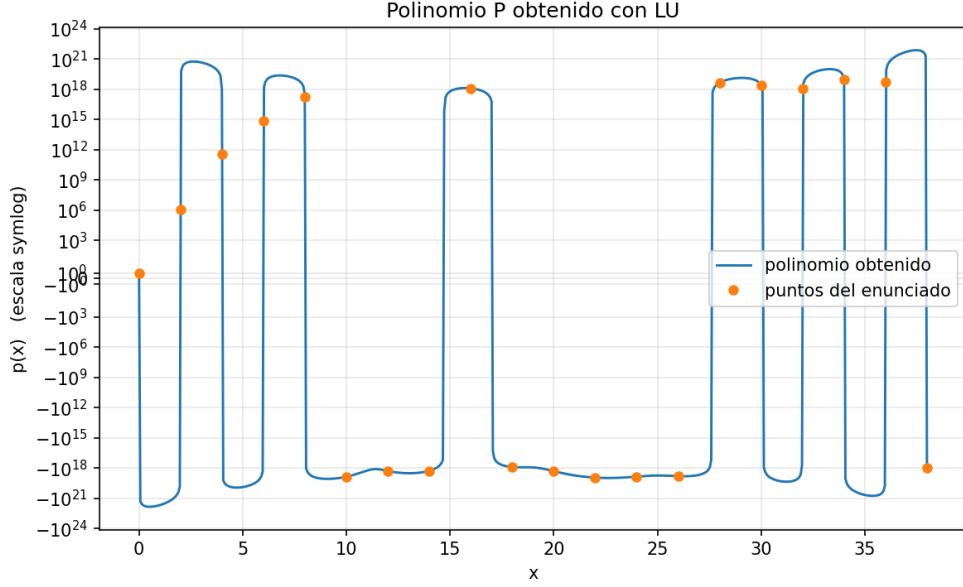


Figura 1: Polinomio P obtenido mediante factorización LU.

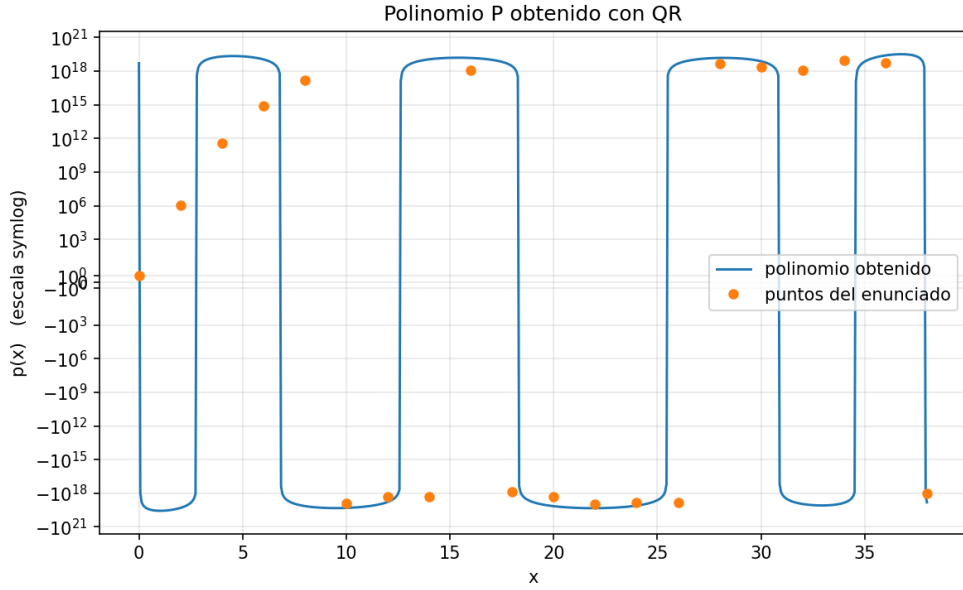


Figura 2: Polinomio P obtenido mediante factorización QR.

En la gráfica se puede observar que el polinomio obtenido sí pasa por todos los puntos dados en el enunciado, por lo que cumple con la condición de interpolar los datos. Sin embargo, entre un punto y otro presenta oscilaciones extremadamente grandes, alcanzando valores cercanos a $\pm 10^{21}$ y cambiando de signo varias veces.

Esto indica que, aunque matemáticamente el polinomio pasa por todos los puntos, no resulta útil para estimar valores intermedios. De hecho, cualquier aproximación entre dos datos produciría resultados exageradamente grandes y sin interpretación razonable dentro del contexto del problema.

La gráfica también permite confirmar visualmente el inconveniente detectado. El polinomio original, $p(x) = 1 + x + x^2 + \dots$, es siempre positivo y presenta un comportamiento creciente. En

cambio, el polinomio obtenido cambia de signo repetidamente, lo que evidencia que no se está recuperando el polinomio original, sino el que interpola exactamente unos datos que ya contenían errores.

Para poder visualizar adecuadamente la gráfica fue necesario utilizar una escala `symlog` en el eje vertical, ya que en escala lineal las diferencias de magnitud impedirían apreciar el comportamiento de la curva. El hecho de requerir esta escala es otra señal de lo desproporcionados que son los valores obtenidos.

Al comparar ambas figuras se observa que el comportamiento general es el mismo, pero con una diferencia importante: la curva obtenida con LU sí pasa por todos los puntos del enunciado, mientras que la obtenida con QR se aparta visiblemente de varios de ellos. Esto concuerda con los residuos medidos, pues el de QR resulta unas cien veces mayor que el de LU ($5,57 \times 10^{19}$ frente a $4,86 \times 10^{17}$), y con la pérdida de ortogonalidad de Q señalada anteriormente.

Se distinguen entonces dos efectos superpuestos. Por un lado, el desbordamiento de los datos de entrada afecta por igual a ambos métodos, y es la razón de que ninguno recupere el polinomio original. Por otro, el proceso de Gram–Schmidt clásico añade su propio error, hasta el punto de que la interpolación deja de cumplirse. La comparación de las dos gráficas permite separar visualmente lo que corresponde al problema de lo que corresponde al método.

3. Código y reproducibilidad

El programa se compone de dos archivos que deben estar en la misma carpeta: `datos.py`, con los arreglos del enunciado, e `interpolacion.py`, con la implementación y el menú de ejecución. Requiere Python 3 con `numpy` y `matplotlib`, que se instalan con

```
pip install numpy matplotlib
```

y se ejecuta con `python interpolacion.py`. El menú permite reproducir cada una de las secciones de este informe: diagnóstico previo, resolución de cada caso, tabla comparativa y gráficas.

Ninguno de los métodos se delegó a una librería: la matriz de Vandermonde, el método de Crout, las dos sustituciones y el proceso de Gram–Schmidt están implementados a mano.

4. Contribuciones individuales

El trabajo se repartió por método: cada integrante se encargó del desarrollo matemático, la implementación y el análisis de una de las tres factorizaciones, mientras que la experimentación, la redacción y la revisión se realizaron de manera conjunta.

Cuadro 6: Distribución del trabajo

Ítem	María Lucía Castillo	Juliana González	Miguel Á. Padilla
Desarrollo matemático	QR	LU	PLU
Implementación	Gram–Schmidt y producto $Q^T y$	Método de Crout y sustituciones	Matriz de Vandermonde, evaluación del polinomio y medidas de error
Experimentación	Ejecución y registro del caso de grado 19	Ejecución y registro del caso de grado 199	Verificación de los datos de entrada
Análisis	QR	LU	PLU
Visualización	Interpretación de las figuras	Módulo de gráficas	Selección y montaje de la evidencia
Redacción	Componente teórico	Componente práctico	Tablas y anexos
Revisión	Conjunta	Conjunta	Conjunta

5. Declaración del uso de inteligencia artificial

5.1. Herramienta y propósito

Durante el desarrollo del examen se emplearon herramientas de inteligencia artificial conversacional como apoyo, con tres propósitos:

- **Comprensión conceptual:** aclarar el significado geométrico de la proyección y de la ortonormalidad en el proceso de Gram–Schmidt, así como la relación entre la independencia lineal de las columnas y la diagonal de R .
- **Apoyo en la redacción:** revisar la precisión y la coherencia de las respuestas del componente teórico, ya escritas por el grupo.
- **Apoyo en la implementación:** organizar el programa y documentar el código.

5.2. Prompts utilizados

Las capturas de los prompts empleados por cada integrante se recogen en el Anexo A. Se declara adicionalmente el siguiente prompt, empleado para la elaboración de este documento:

«Solicitamos la generación del informe en L^AT_EX para Overleaf a partir del desarrollo que el grupo había redactado previamente, adjuntando dicho desarrollo, el enunciado del examen y la rúbrica de evaluación.»

5.3. Elementos aceptados, modificados o rechazados

Se rechazaron:

- El uso de funciones de librería para resolver los sistemas (`lu_factor` de SciPy, `qr` de NumPy). Se implementaron los métodos a mano, ya que el objetivo del examen es dominar el procedimiento y no obtener el resultado.
- El esquema de Horner para evaluar el polinomio, por no corresponder a lo visto en clase. Se reemplazó por la suma directa $a_0 + a_1x + a_2x^2 + \dots$, tal como aparece en el enunciado.
- Explicaciones y desarrollos que excedían lo preguntado, con el fin de no incluir contenido fuera del alcance del curso.

Se modificaron:

- La redacción de varias respuestas, para ajustarla al nivel y al estilo del grupo y reducir la notación innecesaria.
- El alcance de la respuesta a la pregunta 4, precisando qué se entiende por «modificación a la matriz original» antes de responder.
- La forma de marcar la matriz A_4 en la tabla de aplicabilidad. Inicialmente se había señalado únicamente a LU como inestable, pero al verificar se concluyó que el fenómeno es del mal condicionamiento de la matriz y afecta por igual a los tres métodos, por lo que la marca se aplicó a los tres.

Se aceptaron, tras verificarlos:

- La estructura general del programa y la documentación del código.
- El diagnóstico previo de los datos de entrada, una vez comprobado por cuenta propia.

5.4. Verificación y aprendizajes

Las respuestas obtenidas no se dieron por válidas sin comprobación. Los procedimientos de verificación empleados fueron:

- **Comprobación de las identidades de la factorización.** Se verificó que $Q^T Q = I$ y que $QR = A$ para un ejemplo de 3×3 resuelto a mano, y que el producto de la diagonal de R coincide con $|\det(A)|$.
- **Recálculo con aritmética exacta.** Los valores y_i del enunciado se recalcularon usando enteros de Python, que no tienen límite de tamaño, y se compararon con los entregados. Así se determinó exactamente cuántos estaban corrompidos y desde cuál.
- **Revisión del alcance del código.** Se auditó el programa función por función para confirmar que no empleara métodos ajenos al temario, lo que llevó a retirar el esquema de Horner y a documentar explícitamente cada instrucción de librería utilizada.

Aprendizajes. El principal aprendizaje del examen es que la corrección de un método no garantiza la corrección del resultado. Los dos algoritmos implementados resolvieron adecuadamente el sistema que se les entregó, pero ese sistema no era el correcto: los datos ya venían afectados por un desbordamiento y la matriz estaba mal condicionada. Esto permitió comprender en la práctica la diferencia entre un problema mal condicionado, que es una propiedad de la matriz, y un método inestable, que es una propiedad del algoritmo, y observar que ningún método corrige lo primero.

Asimismo, la medición $\|Q^T Q - I\| = 10,46$ permitió comprobar experimentalmente algo que en la teoría aparece como una advertencia: que el proceso de Gram-Schmidt clásico pierde la ortogonalidad de Q cuando las columnas de partida son casi dependientes.

A. Anexo A. Capturas de los prompts utilizados

Juliana González Sánchez

Propiedades de las matrices invertibles Explicando las propiedades de las matrices invertibles. Vale, este es un tema que conecta directo con lo que vimos de "invertibilidad y solución única", así que vamos a construirlo bien. ¿Qué significa que A sea invertible? Significa que existe una matriz A^{-1} tal que $A \cdot A^{-1} = A^{-1} \cdot A = I$ (la matriz identidad). Solo las matrices cuadradas pueden ser invertibles — no tiene sentido hablar de la inversa de una matriz rectangular en este sentido clásico (para eso existen las pseudoinversas, pero es otro tema). Las propiedades — y por qué son todas "la misma cosa" vista distinto Lo importante para tu parcial no es memorizar una lista suelta, sino entender que todas estas condiciones son equivalentes: si se cumple una, se cumplen todas las demás. Esto se llama a veces el "teorema de la matriz invertible".	Para una matriz A cuadrada de tamaño $n \times n$, las siguientes afirmaciones son equivalentes: 1. A es invertible (existe A^{-1}). 2. $\det(A) \neq 0$. El determinante distinto de cero es la forma más rápida de chequear invertibilidad a mano. 3. $\text{rank}(A) = n$ (rango completo). Es decir, ninguna fila ni columna es "redundante" — no se puede escribir como combinación lineal de las demás. 4. Las columnas de A son linealmente independientes. Ninguna columna se puede obtener combinando las otras. 5. Las filas de A son linealmente independientes. Mismo argumento, por filas. 6. El único vector x que satisface $Ax = 0$ es $x = 0$ (núcleo trivial, o "kernel trivial"). Es decir, A no "colapsa" ningún vector distinto de cero al vector cero. 7. Para todo b, el sistema $Ax = b$ tiene solución única. Esta es justo la que conecta con tu primera pregunta del parcial. 8. 0 no es un valor propio (eigenvalue) de A. Si 0 fuera valor propio, existiría un vector propio $v \neq 0$ con $Av = 0$, lo cual contradice la propiedad 6. 9. A se puede llevar a la matriz identidad mediante eliminación de Gauss (o Gauss-Jordan), sin que aparezca ninguna fila de puros ceros. 10. En la eliminación de Gauss, todos los pivotes son distintos de cero (aunque quizás necesites reordenar filas — pivoteo — para lograrlo). Esta es la propiedad que conecta directamente con cuándo puedes aplicar LU sin modificaciones.
¿Cómo se relaciona la invertibilidad a los valores de la diagonal de una matriz? Depende de qué matriz estés hablando. La relación general es esta: Para una matriz triangular Si tienes: $A = \begin{pmatrix} 2 & 4 & 1 \\ 0 & 3 & 5 \\ 0 & 0 & 6 \end{pmatrix}$ el determinante de una matriz triangular es el producto de su diagonal: $\det(A) = 2 \cdot 3 \cdot 6 = 36$ Como: $\det(A) \neq 0$ entonces A es invertible. Por eso, para una matriz triangular: A es invertible $\iff$ ningún elemento de su diagonal es 0	Sabemos que LU incorpora una Matriz triangular inferior L, una matriz triangular superior U, una estructura de 1 en la diagonal de U, y que estas matrices se resuelven mediante sustitución y no eliminación. Esto qué ganancia o beneficio numérico puede presentar? ¿Qué ganancia numérica ofrece que L y U sean triangulares, con estructura de 1 en la diagonal de U, y se resuelvan por sustitución en lugar de eliminación? La estructura triangular de L y U es lo que permite resolver ambos sistemas por sustitución en lugar de eliminación. En un sistema triangular, cada ecuación aporta exactamente una incógnita nueva: en L (triangular inferior), la primera ecuación ya tiene una sola incógnita, así que se despeja directamente y se sustituye hacia abajo (sustitución hacia adelante); en U (triangular superior), es la última ecuación la que trae una sola incógnita, por lo que se despeja y se sustituye hacia arriba (sustitución hacia atrás). Esto evita tener que seguir haciendo operaciones de eliminación fila por fila para simplificar el sistema, ya que la estructura ya está lo más simple posible. Esto tiene una consecuencia directa en el costo computacional: resolver un sistema triangular por sustitución cuesta aproximadamente $O(n^2)$ operaciones, mientras que resolver un sistema general mediante eliminación de Gauss completa cuesta aproximadamente $O(n^3)$ operaciones. La ganancia se vuelve especialmente relevante cuando se debe resolver $Ax=b$ para varios vectores b distintos con la misma matriz A: la factorización $A=LU$ se calcula una sola vez, a un costo de $O(n^3)$, pero una vez obtenida, cada sistema adicional con un b nuevo solo requiere las dos sustituciones ($Ly=b$ y luego $Ux=y$), a un costo de $O(n^2)$ cada una. Esto evita repetir la eliminación completa cada vez que cambia b, que sería mucho más costoso si se tuviera que rehacer para cada sistema. Sobre la estructura de 1 en la diagonal de U (convención del método de Crout): esta no aporta una ganancia de velocidad en sí misma, sino que garantiza que la factorización $A=LU$ sea única. Sin fijar una de las dos diagonales, existirían infinitas formas de repartir los valores
Sobre la estructura de 1 en la diagonal de U (convención del método de Crout): esta no aporta una ganancia de velocidad en sí misma, sino que garantiza que la factorización $A=LU$ sea única. Sin fijar una de las dos diagonales, existirían infinitas formas de repartir los valores entre L y U que multiplicadas igual reconstruyen A (por ejemplo, escalando una fila de L por un factor y la fila correspondiente de U por su inverso). Fijar la diagonal de U en 1 elimina esa ambigüedad y define un único resultado en cada paso del algoritmo.	Explica qué es el condicionamiento de matrices ? Claro. El condicionamiento de una matriz básicamente mide qué tan sensible es la solución de un sistema $Ax = b$ frente a pequeños errores o cambios en los datos. La idea intuitiva Supón que tienes: $Ax = b$ y encuentras una solución x. Ahora imagina que por errores de medición, redondeo o cálculos numéricos, b cambia un poquito: $b \rightarrow b + \Delta b$ Si ese pequeño cambio produce un cambio pequeño en x, el sistema está bien condicionado. Pero si un cambio pequeñísimo en b produce un cambio enorme en x, está mal condicionado. Entonces tenemos: Bien condicionada: pequeño error en b $\rightarrow$ pequeño error en x Mal condicionada: pequeño error en b $\rightarrow$ gran error en x

¿Cómo se mide?

Se utiliza el **número de condición** de la matriz:

$$\kappa(A) = \|A\| \|A^{-1}\|$$

No necesitas pensar que esto significa que tenemos que calcular A^{-1} explícitamente; es una definición matemática del número de condición.

La interpretación importante es:

- $\kappa(A) \approx 1 \rightarrow$ matriz bien condicionada.
- $\kappa(A)$ grande $\rightarrow$ matriz mal condicionada.
- $\kappa(A) \rightarrow \infty \rightarrow$ la matriz se acerca a ser singular.

¿Por qué importa en métodos como LU?

Porque LU es un **método numérico**. Las computadoras trabajan con precisión finita y, por tanto, aparecen errores de redondeo.

Si A está **mal condicionada**, esos pequeños errores pueden amplificarse y hacer que la solución calculada sea muy diferente de la solución exacta.

Por ejemplo, conceptualmente:

$$\text{error de los datos} = 0.001$$

pero una matriz muy mal condicionada podría producir:

$$\text{error en la solución} = 10$$



Componente 2: el "20"

Este es el **valor verdadero o esperado**. Recuerda que el polinomio interpolante, por definición, debe satisfacer $P(x_i)=y_i$ para cada uno de los puntos originales con los que lo construiste. Mirando tu conjunto de datos x20, el segundo punto es $x=1$ con $y=1048575...$ espera, en realidad hay que revisar: el 20 no es un dato aleatorio, es el valor de y correspondiente al punto donde $x=1$ en tu conjunto de datos original (esto viene dado en el enlace de Colab que menciona el ejercicio). Si tu polinomio interpolante fuera calculado con error cero, evaluarlo en $x=1$ debería reproducir exactamente ese valor original — así es como se define un polinomio interpolante: pasa exactamente por los puntos de tus datos.

Componente 3: $P(1) - 20$

Esta es la **diferencia** entre lo que tu método calculó ($P(1)$) y lo que debería haber dado en teoría (20, el valor real del dato). Si tu método fuera perfectamente exacto, esta diferencia sería cero. Cualquier valor distinto de cero es la manifestación numérica del error acumulado durante la factorización y la resolución del sistema — exactamente el tipo de error que predijiste en tu diagnóstico, causado por el mal condicionamiento de la matriz de Vandermonde.

Componente 4: las barras |·| (valor absoluto o norma)

Las barras dobles aquí representan tomar el **valor absoluto** de esa diferencia (en este caso, como $P(1)$ y 20 son números escalares, no vectores, la "norma" se reduce simplemente a valor absoluto). Esto es porque no te interesa si el error fue "de más" o "de menos" — solo qué tan grande es la magnitud del error, sin importar el signo. Si $P(1)$ dio 22, el error absoluto es $|22-20|=2$. Si $P(1)$ dio 18, el error absoluto también es $|18-20|=2$.

Componente 5: dividir entre 20

Puedes explicarme qué significa cada componente del cálculo del valor relativo, que te muestro en la imagen?

Claro, vamos a desmenuzar esta fórmula pieza por pieza. Es el **error relativo**, una forma estándar de medir qué tan lejos está un valor calculado de su valor verdadero, expresado como una proporción en lugar de un número absoluto.

La fórmula: $|P(1) - 20| / 20$

Componente 1: $P(1)$

Esto es evaluar tu polinomio interpolante P (el de grado 19) en $x=1$, usando los coeficientes que obtuviste al resolver el sistema $Va=y$ con LU o con QR. Es decir, una vez tienes $a_0, a_1, \dots, a_{19}$ (los coeficientes calculados numéricamente, con todo y el error de redondeo que puedan traer), sustituyes $x=1$ en:

$$P(1) = a_0 + a_1(1) + a_2(1)^2 + \dots + a_{19}(1)^{19} = a_0 + a_1 + a_2 + \dots + a_{19}$$

Como cualquier potencia de 1 sigue siendo 1, $P(1)$ es simplemente la suma de todos los coeficientes. Este es tu **valor calculado** — la aproximación que produjo tu método numérico.

Componente 2: el "20"

Componente 5: dividir entre 20

Aquí está la diferencia clave entre **error absoluto** y **error relativo**. Si solo miraras $|P(1)-20|=2$, ese número por sí solo no te dice mucho — ¿es un error grande o pequeño? Depende de la escala del problema. Un error de 2 unidades es enorme si el valor esperado era 3, pero es insignificante si el valor esperado era 3 millones. Al dividir entre 20 (el valor verdadero), conviertes ese error absoluto en una **proporción**: te dice qué fracción del valor original representa el error. Por ejemplo, si el error absoluto es 2 y divides entre 20, el error relativo es 0.1, es decir, un 10% de desviación respecto al valor esperado.

Juntando todo:

$$|P(1) - 20| / 20 = |\text{valor calculado} - \text{valor verdadero}| / |\text{valor verdadero}|$$

Esto te da un número adimensional (sin unidades) que representa qué tan grande es el error **en proporción** al tamaño del valor que intentabas reproducir. Es mucho más informativo que el error absoluto solo, porque te permite comparar de forma justa el error del caso de 20 puntos (dividido entre 20) con el error del caso de 200 puntos (dividido entre 200) — aunque las escalas de los problemas sean muy distintas.

Por qué esto es exactamente lo que necesitas para tu diagnóstico

Este error relativo es la métrica que va a confirmar (o refutar) numéricamente lo que predijiste en la parte teórica: si el mal condicionamiento de Vandermonde realmente corrompió la solución, vas a ver un error relativo grande — posiblemente mucho mayor en el caso Q (grado 199) que en el caso P (grado 19), tal como anticipaste que el condicionamiento sería peor con exponentes y valores de x más grandes. Un error relativo cercano a 0 indicaría que, a pesar de tus predicciones, el método logró mantenerse numéricamente estable; un error relativo grande (mayor a 0.01, 0.1, o incluso mayor a 1) confirmaría que el mal condicionamiento efectivamente arruinó la precisión del resultado.

Miguel Ángel Padilla Rosero

Compara las ventajas de LU y PLU en computación científica estructurando la respuesta en dos apartados. Analiza el costo asintótico de la factorización (FLOPs) frente a resolver sistemas con múltiples vectores b ($O(n^2)$), el impacto nulo en FLOPs de permutar filas por índices, el control de errores numéricos ($|l_{ij}| \leq 1$) y por qué PLU es el estándar en librerías como LAPACK (dgesv), SciPy y MATLAB.

1. Complejidad computacional, coste en FLOPs y amortización con múltiples lados derechos

La factorización directa mediante eliminación gaussiana requiere exactamente el mismo coste asintótico en operaciones de punto flotante tanto en su versión básica ($A = LU$) como con pivoteo parcial ($PA = LU$):

FLOPs_{factorización} $\approx \frac{2}{3}n^3 + O(n^2)$

Operación	Factorización ($A \rightarrow LU$ o $PA \rightarrow LU$)	Permutación de filas (P)	Resolución por vector b (Ax)
Coste Asintótico	$O(n^3)$ ($\approx \frac{2}{3}n^3$ FLOPs)	$O(n^2)$ comparaciones, $O(n)$ swaps	$O(n^2)$ ($2n^2$)

El estándar de la industria (LAPACK, SciPy, MATLAB):

Rutinas nucleares como `dgesv` y `dgetrf` en LAPACK, el operador `\` y `lu` en MATLAB, y `scipy.linalg.solve` o `scipy.linalg.lu` en SciPy implementan PLU de forma predeterminada para sistemas densos no simétricos.

Dado que la penalización computacional de permutar índices es nula y el beneficio en estabilidad es absoluto, el software científico moderno nunca ejecuta LU sin pivoteo a menos que la matriz posea propiedades estructurales garantizadas que prevengan la inestabilidad (como matrices simétricas definidas positivas mediante Cholesky LL^T , o matrices diagonalmente dominantes estrictas).

¿Quieres ver cómo varía la estabilidad comparando pivoteo parcial frente a pivoteo completo?

Si

1. Tabla Comparativa Resumen			
Matriz	Clasificación y Determinante	LU (sin pivoteo)	PLU (pivoteo parcial)
$A_1 = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}$	Regular / Bien condicionada $\det(A_1) = 2 \neq 0$	Si	Si
$A_2 = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$	Singular / Rango deficiente $\det(A_2) = 0$ (Rango = 1)	No	No
$A_3 = \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}$	Regular con pivote nulo $\det(A_3) = -1 \neq 0$	No	Si
$A_4 = \begin{bmatrix} 1 & 1 \\ 1 & 1 + 10^{-12} \end{bmatrix}$	Regular, pero Casi-singular $\det(A_4) = 10^{-12} \approx 0$ $\kappa(A_4) \approx 4 \times 10^{12}$	Problemático	Problemático

Impacto nulo de las permutaciones en FLOPs: El pivoteo parcial no ejecuta sumas ni multiplicaciones de punto flotante al permutar. Solo realiza comparaciones lógicas para encontrar el máximo en valor absoluto de la columna activa ($\approx \frac{1}{2}n^2$ comparaciones) y reordena punteros o índices en un vector de permutaciones de tamaño n (típicamente denominado `ipiv`). El coste de intercambio en memoria es insignificante frente a las $\approx \frac{2}{3}n^3$ operaciones aritméticas de los bucles de eliminación.

Amortización con múltiples vectores b: Factorizar la matriz desacopla el coste cúbico del sistema. Para resolver $Ax_k = b_k$ con m vectores independientes, el coste es:

- Permutación y sustitución progresiva ($Ly = Pb_k$): n^2 FLOPs.
- Sustitución regresiva ($Ux = y$): n^2 FLOPs.

El coste marginal por cada b_k adicional es de solo $2n^2$ FLOPs ($O(n^2)$), permitiendo resolver miles de sistemas con la misma matriz sin recalcular la descomposición.

2. Estabilidad numérica, control del error y adopción en librerías científicas

La descomposición $A = LU$ pura es numéricamente inestable para matrices generales y falla catastróficamente si algún elemento diagonal $a_{kk}^{(k-1)}$ es nulo o cercano a cero. La descomposición $PA = LU$ resuelve esta debilidad estructural:

Acotamiento de multiplicadores ($|l_{ij}| \leq 1$): En cada paso k , PLU selecciona como pivote el elemento de mayor magnitud de la subcolumna restante, $p = \operatorname{argmax}_{i \geq k} |a_{ik}^{(k-1)}|$, intercambiando las filas k y p . Esto garantiza que los multiplicadores de Gauss satisfagan estrictamente:

$A_0 = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$
 $A_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 + 10^{-12} \end{bmatrix}$

Con la siguiente imagen haz una tabla comparativa, entre si pueden ser o no resueltos los sistemas, con las factorizaciones LU, PLU, y QR y porque explíatelo en la explicación

A continuación se presenta un análisis comparativo y detallado sobre la aplicabilidad y estabilidad de las factorizaciones LU (sin pivoteo), PLU (pivoteo parcial) y QR para resolver el sistema lineal $A_0 x = b$ correspondiente a cada una de las matrices de la imagen.

2. Análisis Detallado Matriz por Matriz

Matriz $A_1 = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}$

Determinante: $\det(A_1) = (2)(3) - (1)(4) = 2 \neq 0$. Las filas y columnas son linealmente independientes; la matriz es regular e invertible.

Comportamiento con LU (sin pivoteo):

- El pivote inicial es $a_{11} = 2 \neq 0$.
- Multiplicador gaussiano: $l_{21} = \frac{4}{2} = 2$.
- Operación de eliminación: $F_2 \leftarrow F_2 - 2F_1 \implies [4, 3] - 2[2, 1] = [0, 1]$.
- Matrices resultantes:
$$L = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

Como todos los elementos diagonales de U son no nulos ($u_{11} = 2, u_{22} = 1$), el sistema se resuelve directamente mediante sustitución progresiva ($Ly = b$) y regresiva ($Ux = y$).

Observación numérica: El multiplicador $|l_{21}| = 2 > 1$. En dimensiones mayores, multiplicadores mayores a 1 pueden propagar errores de redondeo, aunque en este caso 2×2 no genera problema.

Comportamiento con PLU:

En la primera columna, el algoritmo compara $|a_{11}| = 2$ frente a $|a_{21}| = 4$. Al ser $4 > 2$, permuta las filas 1 y 2 mediante la matriz de permutación $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.

El nuevo pivote pasa a ser 4. El multiplicador es $l_{21} = \frac{2}{4} = 0.5 \leq 1$.

Factorización:

19

- Comportamiento con QR:
 - Al tener columnas no nulas y linealmente independientes, la factorización ortogonal-triangular $A_1 = QR$ existe y es única (con diagonal de R positiva).
 - R resulta triangular superior con diagonal estrictamente positiva ($r_{11} > 0, r_{22} > 0$). El sistema $Rx = Q^T b$ se resuelve mediante sustitución regresiva con estabilidad hacia atrás garantizada.

Matriz $A_2 = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$

- Determinante: $\det(A_2) = (1)(4) - (2)(2) = 0$. La segunda fila es exactamente el doble de la primera ($F_2 = 2F_1$). El rango de la matriz es 1.
- Comportamiento con LU y PLU:
 - En LU: Con pivote $a_{11} = 1$, el multiplicador es $l_{21} = \frac{2}{1} = 2$. La fila resultante en U es:

$$F_2 - 2F_1 = [2, 4] - 2[1, 2] = [0, 0]$$

$$U = \begin{bmatrix} 1 & 2 \\ 0 & 0 \end{bmatrix}$$

Matriz $A_3 = \begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}$

- Determinante: $\det(A_3) = (0)(2) - (1)(1) = -1 \neq 0$. La matriz es estrictamente invertible, simétrica e indefinida.
- Comportamiento con LU (sin pivoteo):
 - Colapso inmediato: El algoritmo necesita calcular el multiplicador de la primera columna:

$$l_{21} = \frac{a_{21}}{a_{11}} = \frac{1}{0}$$
 - La división por cero aborta la ejecución antes de iniciar cualquier cálculo.
 - Fundamento algebraico: La descomposición $A = LU$ existe y es única si y solo si todos los menores principales de orden $k < n$ son no nulos ($\Delta_k \neq 0$). En este caso, el primer menor principal es $\Delta_1 = \det([0]) = 0$. Por tanto, la **factorización LU pura no existe** para A_3 .

Matriz $A_4 = \begin{bmatrix} 1 & 1 \\ 1 & 1 + 10^{-12} \end{bmatrix}$

- Determinante y Condicionamiento:

$$\det(A_4) = 1(1 + 10^{-12}) - 1(1) = 10^{-12} \neq 0$$
 - Teóricamente es invertible. Sin embargo, sus dos filas (y columnas) son casi colineales.
 - El número de condición en norma espectral es del orden de:

$$\kappa(A_4) \approx \frac{4}{10^{-12}} = 4 \times 10^{12}$$
- Esto significa que la matriz está **severamente mal condicionada (ill-conditioned)**. En aritmética de punto flotante estándar (IEEE 754):
 - En **precisión simple** (`float32`, con $\epsilon_{mach} \approx 1.19 \times 10^{-7}$), el valor $1 + 10^{-12}$ se redondea a 1, volviendo la matriz numéricamente singular en máquina ($\det = 0$).
 - En **precisión doble** (`float64`, con $\epsilon_{mach} \approx 2.22 \times 10^{-16}$), el sistema es resoluble pero se pierden cerca de 12 dígitos significativos de precisión en la solución calculada frente a cualquier perturbación en b .
- Comportamiento con LU (sin pivoteo):
 - Pivote $a_{11} = 1$. Multiplicador: $l_{21} = \frac{1}{1} = 1$.
 - Eliminación en la segunda fila:

$$u_{22} = (1 + 10^{-12}) - (1)(1) = 10^{-12}$$

$$L = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 1 \\ 0 & 10^{-12} \end{bmatrix}$$

+ Preguntarle a Gemini

Flash Extendido

- En PLU: Si se permuta buscando el pivote máximo ($|2| > |1|$), se obtiene:

$$PA_2 = \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix} \rightarrow l_{21} = 0.5 \rightarrow U = \begin{bmatrix} 2 & 4 \\ 0 & 0 \end{bmatrix}$$

- En ambos casos, el elemento diagonal $u_{22} = 0$.
- ¿Por qué no se puede resolver? En la fase de sustitución regresiva para hallar x_2 , se requiere calcular $x_2 = \frac{b_2}{u_{22}} = \frac{0}{0}$, produciendo una división por cero. Si $y_2 \neq 0$, el sistema es incompatible; si $y_2 = 0$, el sistema tiene infinitas soluciones. Ningún algoritmo estándar de resolución directa puede entregar una solución única.
- Comportamiento con QR:
 - El proceso de ortogonalización (Gram-Schmidt o Householder) detecta la dependencia lineal. Al proyectar la segunda columna sobre la primera, el vector ortogonal residual es idénticamente nulo:

$$v_2 = a_2 - \text{proj}_{v_1}(a_2) = \begin{bmatrix} 2 \\ 4 \end{bmatrix} - 2 \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
 - La matriz R contendrá un cero en su diagonal ($r_{22} = 0$). Al intentar resolver $Rx = Q^T b$, la sustitución regresiva falla por división por cero. Para sistemas singulares como este se requiere regularización, rango truncado (Rank-Revealing QR) o la pseudoinversa de Moore-Penrose mediante SVD.

- Comportamiento con PLU:
 - El pivoteo parcial inspecciona la primera columna: $|a_{11}| = 0$ y $|a_{21}| = 1$.
 - Al ser $1 > 0$, el algoritmo intercambia la fila 1 con la fila 2 ($P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$).
 - La matriz permutada queda:

$$PA_3 = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

- Como ya tiene ceros debajo del nuevo pivote, el multiplicador es simplemente $l_{21} = 0$.
- Factorización resultante:

$$L = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I, \quad U = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

- Ambas matrices son no singulares ($u_{11} = 1, u_{22} = 1$), resolviendo el sistema de forma trivial y exacta.
- Comportamiento con QR:
 - Los métodos ortogonales no operan dividiendo por el elemento diagonal original a_{11} . Construyen reflectores de Householder o rotaciones de Givens basándose en la norma euclidiana del vector columna completo $\|a_1\|_2 = \sqrt{0^2 + 1^2} = 1$.
 - La matriz Q resultante es una matriz ortogonal bien definida (en este caso una simple permutación/reflexión) y R es no singular. El sistema se resuelve sin perturbaciones numéricas.

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

3. Conclusiones Principales

- LU sin pivoteo** es el método más restrictivo: falla tanto por singularidad (A_2) como por la presencia de ceros en posiciones pivote intermedias (A_3), aun cuando la matriz sea perfectamente invertible.
- PLU (pivoteo parcial)** resuelve con éxito cualquier matriz invertible bien condicionada (A_1, A_3), reordenando filas para esquivar ceros y acotar los multiplicadores ($|l_{ij}| \leq 1$), pero no puede resolver matrices singulares (A_2) ni eliminar el error en sistemas mal condicionados (A_4).
- QR** ofrece la máxima estabilidad hacia atrás: maneja ceros estructurales de forma nativa sin permutar (A_3) y minimiza la amplificación del error de redondeo en matrices al borde de la singularidad (A_4), requiriendo únicamente rango completo para garantizar unicidad en la sustitución regresiva.

🔄 🔍 🗑️ ⋮

Más allá de garantizar una solución única para $Ax = b$, ¿qué rol juega la invertibilidad en las factorizaciones LU y PLU? Explica por qué ser invertible no basta para que exista LU clásico (menores principales no nulos) frente a PLU, y qué otras ventajas algebraicas se derivan (como el cálculo del determinante y la detección de pivotes nulos en U)

La invertibilidad de una matriz $A \in \mathbb{R}^{n \times n}$ tal que $\det(A) \neq 0$ es la condición matemática que asegura un mapeo biyectivo y la existencia de una solución única para $Ax = b$. Sin embargo, en el ámbito de las factorizaciones matriciales directas, la invertibilidad no es suficiente ni equivalente a la existencia de la descomposición LU clásica.

A continuación se profundiza en el rol estructural de la invertibilidad, las condiciones algebraicas de existencia de LU frente a PLU y las ventajas computacionales derivadas.

1. Por qué la invertibilidad no basta para LU: El Teorema de los Menores Principales Líderes

Para que una matriz admita una factorización $A = LU$ (donde L es triangular inferior unitaria con $l_{ii} = 1$ y U es triangular superior no singular), debe poder ejecutarse la eliminación gaussiana clásica sin intercambiar filas.

Fundamento de los bloques y menores principales

Consideremos la partición de orden k de la matriz A :

$$A = \begin{bmatrix} A_k & B_k \\ C_k & D_k \end{bmatrix}$$

donde $A_k \in \mathbb{R}^{k \times k}$ es la submatriz principal líder de orden k . Si existe $A = LU$, entonces particionando L y U de forma compatible:

$$\begin{bmatrix} A_k & B_k \\ C_k & D_k \end{bmatrix} = \begin{bmatrix} L_k & 0 \\ L_{21} & L_{22} \end{bmatrix} \begin{bmatrix} U_k & U_{12} \\ 0 & U_{22} \end{bmatrix} = \begin{bmatrix} L_k U_k & L_k U_{12} \\ L_{21} U_k & L_{21} U_{12} + L_{22} U_{22} \end{bmatrix}$$

De esta igualdad se deduce inmediatamente que:

$$A_k = L_k U_k \quad \forall k \in \{1, 2, \dots, n\}$$

Aplicando determinantes:

$$\det(A_k) = \det(L_k) \cdot \det(U_k) = (1) \cdot \prod_{j=1}^k u_{jj} = \prod_{j=1}^k u_{jj}$$

De aquí se desprenden dos conclusiones críticas:

1. **Condición de existencia estricta:** Para que la factorización exista y U sea regular (ningún pivote $u_{jj} = 0$), es condición necesaria y suficiente que:

$$\Delta_k = \det(A_k) \neq 0 \quad \forall k \in \{1, 2, \dots, n-1\}$$

2. La invertibilidad solo condiciona el final: La afirmación " $\det(A) \neq 0$ " únicamente garantiza que $\det(A_n) \neq 0$. No impone restricción alguna sobre los menores intermedios $\Delta_1, \Delta_2, \dots, \Delta_{n-1}$.

El contraejemplo canónico

Una matriz elemental de permutación o rotación:

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- Es perfectamente invertible: $\det(A) = -1 \neq 0$ y $A^{-1} = A$.
- Sin embargo, su primer menor principal es $\Delta_1 = \det([0]) = 0$.
- Si intentamos $A = LU$:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ l_{21} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{bmatrix} = \begin{bmatrix} u_{11} & u_{12} \\ l_{21}u_{11} & l_{21}u_{12} + u_{22} \end{bmatrix}$$

Para igualar la primera fila requerimos $u_{11} = 0$ y $u_{12} = 1$. Pero entonces el elemento $(2, 1)$ da $l_{21} \cdot u_{11} = l_{21} \cdot 0 = 0 \neq 1$. El sistema no tiene solución para ningún $l_{21} \in \mathbb{R}$.

2. El rol de la invertibilidad en PLU: Existencia Universal

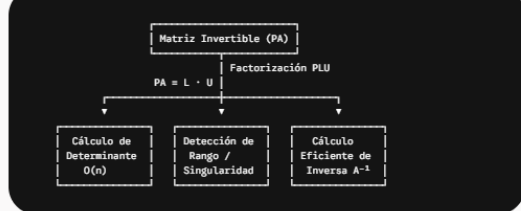
Al introducir una matriz de permutación P , el problema de los menores principales nulos desaparece:

Teorema fundamental de PLU: Si $A \in \mathbb{R}^{n \times n}$ es una matriz invertible ($\text{rango}(A) = n$), siempre existe una matriz de permutación P tal que la matriz reordenada $\tilde{A} = PA$ satisface que todos sus menores principales líderes son no nulos:

$$\det((PA)_k) \neq 0 \quad \forall k \in \{1, \dots, n\}$$

Por ende, toda matriz invertible admite factorización $PA = LU$.

- Mecanismo algebraico:** Si en el paso k el elemento $a_{kk}^{(k-1)} = 0$, la invertibilidad de A asegura que la subcolumna activa no puede ser idénticamente nula:



A. Cálculo eficiente del determinante en $O(n)$ (tras factorizar)

Calcular el determinante por cofactores (Leibniz) tiene un coste inmanejable de $O(n!)$. Con la descomposición $PA = LU$:

$$\det(PA) = \det(P) \det(A) = \det(L) \det(U)$$

Dado que:

1. $\det(L) = \prod_{i=1}^n l_{ii} = 1$ (porque L es triangular con unos en la diagonal).
2. $\det(U) = \prod_{i=1}^n u_{ii}$ (el producto de sus elementos diagonales).
3. $\det(P) = (-1)^s$, donde s es el número total de intercambios de filas efectuados durante la eliminación.

Se obtiene:

$$\det(A) = (-1)^s \prod_{i=1}^n u_{ii}$$

+ Preguntarle a Gemini

Flash



María Lucía Castillo García

Yo sé que por definición se sobreentiende por el tema de la invertibilidad, pero me gustaría dejar explícito que se hace sobre una matriz cuadrada en este párrafo:

GENERAL (Revisar): Si A es invertible, entonces existe un A^{-1} , lo que nos garantiza que el sistema $Ax = b$ tiene solución única $x = A^{-1}b$ para cualquier b . Esto es una consecuencia directa de que el determinante es distinto de 0, lo cual equivale a que las columnas de A sean linealmente independientes. Si A no fuera invertible, el sistema no tendría solución única, podría no tener solución (incompatible) o tener infinitas soluciones (compatible indeterminado).

Mostrar menos ^>

Sí, puedes hacerlo explícito sin que quede forzado. De hecho, ayuda a evitar la duda de qué pasa con matrices rectangulares.

Podrías escribir:

General: Considerando el caso de una matriz cuadrada $A \in \mathbb{R}^{n \times n}$, si A es invertible, entonces existe A^{-1} , lo que garantiza que el sistema $Ax = b$ tiene una única solución dada por $x = A^{-1}b$ para cualquier vector b . Esto es consecuencia de que $\det(A) \neq 0$, lo cual equivale a que las columnas de A sean linealmente independientes. Si A no fuera invertible, el sistema no tendría una solución única: podría no tener solución (sistema incompatible) o tener infinitas soluciones (sistema compatible indeterminado).

O, si lo quieres todavía más natural:

General: Para una matriz cuadrada A , la invertibilidad y la existencia de una solución única están directamente relacionadas. Si A es invertible, existe A^{-1} y el sistema $Ax = b$ tiene una única solución $x = A^{-1}b$ para cualquier b . Esto ocurre porque $\det(A) \neq 0$, lo que equivale a que las columnas de A sean linealmente independientes. Por el contrario, si A no es invertible, el sistema no tendrá solución única: puede no tener solución o tener infinitas soluciones.

Me gusta más la segunda porque empieza precisamente con la idea que pregunta el enunciado: la relación entre invertibilidad y solución única. Luego ya introduces la inversa y el determinante como justificación.

Mira, yo lo redacté de esta manera para el QR en específico, sin embargo, considero que hay cositas que pueden estar incompletas, pero tampoco quiero dar información que no me piden. Me explico, siento que queda muy a la ligera el hecho de que R también sea invertible si A lo es, no porque no sea cierto sino porque parte de esto es consecuencia de que sus columnas sean linealmente independientes, pero como te digo, tampoco quiero responder a un detalle que a lo mejor no se me está exigiendo en la naturaleza de la pregunta....

- QR: Esta misma relación entre invertibilidad y solución única también permite utilizar la factorización QR. Si A es una matriz cuadrada e invertible, entonces puede factorizarse como $A = QR$, donde Q es una matriz ortogonal y R es una matriz triangular superior. Como Q es ortogonal, se cumple que $Q^{-1} = Q^T$. Por ello, el sistema $Ax = b$ puede reescribirse como $QRx = b$. Multiplicando ambos lados por Q^T , se obtiene $Rx = Q^Tb$; y de esta forma el problema se reduce a resolver un sistema triangular superior.

Además, la invertibilidad de A implica que R también es invertible, lo cual se refleja en que sus elementos diagonales son diferentes de 0. Debido a esto, el sistema triangular $Rx = Q^Tb$ puede resolverse mediante sustitución hacia atrás y tiene una única solución. Por tanto, cuando A es invertible, la factorización QR permite encontrar de manera eficiente la solución única del sistema $Ax = b$ sin necesidad de calcular explícitamente la inversa de A .

TAMBIÉN CONSIDERO QUE ESTO LE FALTA DESARROLLAR:
Multiplicando ambos lados por Q^T , se obtiene $Rx = Q^Tb$; y de esta forma el problema se reduce a resolver un sistema triangular superior. Porque claro, se entiende que se reduce a resolver un sistema triangular superior, pero se me hace un poco general o poco claro por qué esto se vuelve positivo en relación a la invertibilidad, no sé si me hago entender.

Por favor, también dime si consideras que hay algún detalle conceptual que a mí se me está pasando por alto también especificar, que a lo mejor no te especificé

Mostrar menos ^>

Sí, te entiendo perfectamente. Y de hecho creo que tu intuición es correcta: no necesitas convertir esta respuesta en una explicación completa de QR, pero sí conviene cerrar un poquito mejor el razonamiento para que no parezca que de repente aparece la afirmación " R es invertible" sin explicar por qué.

Yo distinguí entre lo que la pregunta exige demostrar y lo que simplemente necesitas conectar para que tu argumento sea sólido.

1. El punto que sí falta conectar
Tu texto dice:
"Además, la invertibilidad de A implica que R también es invertible..."

Eso es correcto, pero queda un poco brusco. La conexión que te falta es muy sencilla:

$$A = QR$$

y Q siempre es invertible porque es ortogonal. Por lo tanto, si A es invertible, necesariamente R también debe serlo.

Intuitivamente: en la factorización QR, Q no es la que puede "perder" información porque es invertible. Entonces, si A no perdió información y Q tampoco, R tampoco puede haberla perdido.

Y ahí sí puedes conectar con la diagonal de R :

$$R \text{ invertible} \iff r_{ii} \neq 0 \text{ para todo } i$$

porque R es triangular.

Eso sí me parece necesario, porque precisamente estás hablando de cómo la invertibilidad de A termina garantizando la solución única.

2. Y sí: la parte de Q^T merece una frase más
Estoy de acuerdo contigo. Actualmente tienes:

"Multiplicando ambos lados por Q^T , se obtiene $Rx = Q^Tb$; y de esta forma el problema se reduce a resolver un sistema triangular superior."

Matemáticamente está perfecto, pero la relación con la pregunta queda un poco escondida.

Lo importante es explicar por qué esa transformación es útil:

$$Ax = b$$
$$QRx = b$$
$$Q^TQRx = Q^Tb$$

y como

y como

$$Q^TQ = I,$$

queda

$$Rx = Q^Tb.$$

Entonces, no estamos cambiando el problema ni perdiendo información; simplemente estamos transformándolo en un sistema equivalente que es más sencillo de resolver.

Y aquí aparece la conexión con la solución única:

- $Q^TQ = I \rightarrow$ podemos transformar el sistema sin alterar sus soluciones.
- R es triangular $\rightarrow$ podemos resolverlo mediante sustitución hacia atrás.
- R es invertible $\rightarrow$ sus elementos diagonales son distintos de cero $\rightarrow$ la sustitución hacia atrás puede hacerse en todos los pasos y produce una única solución.

Ese es realmente el hilo que yo haría explícito.

Yo te lo dejaría así
Sin meter Gram-Schmidt, determinantes de Q , residuos, rango, mínimos cuadrados ni nada que la pregunta no te está pidiendo:

QR: Esta misma relación entre invertibilidad y solución única también permite utilizar la factorización QR. Si A es una matriz cuadrada e invertible, puede factorizarse como $A = QR$, donde Q es una matriz ortogonal y R es una matriz triangular superior. Como Q es ortogonal, se cumple que $Q^{-1} = Q^T$. Por ello, el sistema $Ax = b$ puede reescribirse como

ello, el sistema $Ax = b$ puede reescribirse como

$$QRx = b.$$

Multiplicando ambos lados por Q^T , se obtiene

$$Q^TQRx = Q^Tb$$

y, como $Q^TQ = I$,

$$Rx = Q^Tb.$$

De esta manera, el sistema original se transforma en un sistema triangular superior equivalente, sin cambiar su solución. Además, como $A = QR$ y Q es siempre invertible, si A es invertible entonces R también debe serlo. Al ser R triangular superior, esto implica que sus elementos diagonales son diferentes de cero. Por ello, el sistema $Rx = Q^Tb$ puede resolverse mediante sustitución hacia atrás, realizando todas las divisiones necesarias y obteniendo una única solución.

Por tanto, cuando A es invertible, la factorización QR permite encontrar de manera eficiente la solución única del sistema $Ax = b$, sin necesidad de calcular explícitamente la inversa de A .

¿Qué detalles conceptuales creo que ahora sí quedan cubiertos?

1. Que estamos suponiendo A cuadrada.
Ya lo pusiste, y me parece bien.
2. Que Q es invertible.
No basta solamente con decir $Q^T = Q^{-1}$; conviene que quede claro que eso es lo que permite transformar el sistema.
3. Que R es invertible si A lo es.
Este era el pequeño salto que tú misma detectaste. Lo solucionamos con $A = QR \cdot Q$ invertible.
4. Por qué importan los elementos diagonales de R .

4. Por qué importan los elementos diagonales de R .

Porque al ser triangular, R es invertible precisamente cuando ninguno de esos elementos es cero. Y eso garantiza que la sustitución hacia atrás no se quede sin poder dividir.

5. La conexión final con la pregunta.

No solamente "QR resuelve el sistema", sino **por qué la invertibilidad termina produciendo una única solución**.

Y creo que **ahí deberías parar**. No metería en esta respuesta que $r_{jj} = \|v_j\|$, ni Gram-Schmidt, ni rango, ni mínimos cuadrados. Todo eso es verdadero y relevante para estudiar QR, pero **para esta pregunta concreta** empieza a responder cosas que no te están preguntando.

De hecho, me gusta bastante más esta versión porque conserva el paralelismo con tus apartados de LU y PLU, pero en QR queda explícito **dónde entra Q , dónde entra R y dónde exactamente aparece la unidad**.

Mira, te voy a ser sincera, yo redacté esto en un principio:

- QR: Esta misma relación entre invertibilidad y solución única también permite utilizar la factorización QR. Si A es una matriz cuadrada e invertible, entonces puede factorizarse como $A = QR$, donde Q es una matriz ortogonal y R es una matriz triangular superior. Como Q es ortogonal, se cumple que $Q^{-1} = Q^T$. Por ello, el sistema $Ax = b$ puede reescribirse como $QRx = b$. Multiplicando ambos lados por Q^T , se obtiene $Rx = Q^Tb$; y de esta forma el problema se reduce a resolver un sistema triangular superior.

Además, la invertibilidad de A implica que R también es invertible, lo cual se refleja en que sus elementos diagonales son diferentes de 0. Debido a esto, el sistema triangular $Rx = Q^Tb$ puede resolverse mediante sustitución hacia atrás y tiene una única solución.

Por tanto, cuando A es invertible, la factorización QR permite encontrar de manera eficiente la solución única del sistema $Ax = b$ sin necesidad de calcular explícitamente la inversa de A .

Pero hay varias razones por las que no me convencían, porque consideraba que hay cositas que pueden estar incompletas, pero tampoco quiero dar información que no me piden. Me explico, siento que queda muy a la ligera el hecho de que R también sea invertible si A lo es, no porque no sea cierto sino porque parte de esto es consecuencia de que sus columnas sean linealmente independientes, pero como te digo, tampoco quiero responder al detalle cosas que a lo mejor no se me están exigiendo por la naturaleza de la pregunta...

También pensaba que a ESTO LE FALTA DESARROLLO:

Multiplicando ambos lados por Q^T , se obtiene $Rx = Q^Tb$; y de esta forma el problema se reduce a resolver un sistema triangular superior.

Porque claro, se entiende que se reduce a resolver un sistema triangular superior, pero se me hace un poco general o poco claro por qué esto se vuelve positivo en relación a la invertibilidad, no sé si me hago entender.

YO LE dije a Chat, pero no me gustó del todo su cambio. Porque no noto como tanta precisión, PERO y OJO CON ESTO, NO quiero que sea tan técnico porque yo no pienso así con tanta notación, y tampoco quiero que diga información extra que no me piden, ni que sea tan extenso...

Además, no quiero cambiar el tono que se lleva en las respuestas, porque mira cómo va por ahora las respuestas construidas:

- LU: Esta relación entre la invertibilidad y la solución única de un sistema es la que hace posible aplicar factorización LU, donde el objetivo es hallar la solución única de forma eficiente reescribiendo $A = LU$, y resolviendo dos sistemas triangulares $Ly = b$ y $Ux = y$.

Además, en la factorización de Crout, la invertibilidad de (A) se relaciona con que los elementos de la diagonal de (L) , que actúan como pivotes durante el procedimiento, sean diferentes de cero. Esto nos permite realizar las divisiones necesarias para calcular (L) y (U) sin inconvenientes.

- PLU: Aunque el teorema asegura que si A es invertible la solución única existe, en la práctica la eliminación de Gauss directa o LU pura falla si aparece un pivote cero. Sin embargo, precisamente porque A es invertible, se garantiza que siempre existe al menos un elemento no nulo en las filas restantes de esa columna. En consecuencia, siempre es posible encontrar una matriz de permutación P que reordene las filas para completar la eliminación de Gauss, obteniendo:
 $PA = LU$ (o equivalentemente, $A = PTLU$)
Donde P es una matriz de permutación, L es triangular inferior unitaria y U es triangular superior con elementos diagonales no nulos ($u_{ii} \neq 0$). De este modo, la invertibilidad de A asegura que el sistema permutado $PAX = Pb$ se resuelva de forma única y estable en dos pasos triangulares:
 $Ly = Pb$ (sustitución hacia adelante para hallar y)
 $Ux = y$ (sustitución hacia atrás para hallar x)
Recuperando de forma exacta la solución única:
 $x = U^{-1}L^{-1}y$
 $Pb = A^{-1}b$
Espero haberme hecho entender
Mostrar menos

Te hiciste entender perfectamente, y tus dos instintos eran correctos. Te digo qué era exactamente lo que faltaba en cada caso:

En lo de **R invertible** — no era que estuviera mal, es que estaba *afirmado* y no *justificado*. Se arregla con una sola oración intermedia: Invertible $\Leftrightarrow$ columnas LI $\Leftrightarrow$ ningún r_{ii} se anula.

En lo del **sistema triangular** — aquí diste con algo fino. Tu párrafo decía *qué* pasa (se reduce a un triangular) pero *no por qué eso conecta con la invertibilidad*. Y la conexión es la misma que tú ya usaste en LU: la **sustitución hacia atrás divide entre los elementos de la diagonal**, y la invertibilidad es justo lo que permite esas divisiones. Es tu propia frase de LU — "nos permite realizar las divisiones necesarias" — aplicada a QR.

Aquí va tu texto con esos dos arreglos, sin cambi^{ar} nada ni agregar temas nuevos:

QR: Esta misma relación entre la invertibilidad y la solución única es la que permite aplicar la factorización QR. Si A es una matriz cuadrada e invertible, puede factorizarse como $A = QR$, donde Q es una matriz ortogonal y R es triangular superior. Como Q es ortogonal, se cumple que $Q^{-1} = Q^T$, de modo que el sistema $Ax = b$ se reescribe como $QRx = b$ y, al multiplicar ambos lados por Q^T , queda reducido a $Rx = Q^Tb$. Conviene notar que este primer paso no impone ninguna condición sobre A : la inversa de Q existe siempre y se obtiene sin necesidad de cálculo alguno, simplemente transponiendo.

Es en el segundo paso donde interviene la invertibilidad. Como A invertible equivale a que sus columnas sean linealmente independientes, y cada elemento diagonal de R mide el aporte independiente de la columna correspondiente, dicha independencia garantiza que ninguno de ellos se anule ($r_{ii} \neq 0$). En consecuencia R también es invertible, y el sistema triangular $Rx = Q^Tb$ puede resolverse mediante sustitución hacia atrás: en cada fila se despeja una incógnita dividiendo entre el elemento diagonal correspondiente, división que solo es posible porque dichos elementos son distintos de cero.

Por tanto, cuando A es invertible, la factorización QR permite hallar la solución única de $Ax = b$ de manera eficiente y sin calcular explícitamente A^{-1} , pues la inversa de Q se reduce a su transpuesta y la de R se evita mediante la sustitución hacia atrás.

Qué cambió, en concreto

Son tres cosas, todo lo demás es tuyo tal cual:

1. Agregué la frase de que el primer paso *no exige nada de A*. Esto no es relleno — es lo que le da precisión a tu respuesta, porque **aísla dónde actúa la invertibilidad**. Sin eso, el lector no sabe si la condición se necesita en el paso de Q^T , en el de R , o en los dos.
2. La justificación de R invertible pasó de afirmación a consecuencia, en una sola oración.
3. La sustitución hacia atrás ahora dice *por qué* la invertibilidad importa ahí: por las divisiones. Eso además te deja el paralelismo con tu párrafo de LU, que va a leerse muy bien.

Sobre lo que decidiste no incluir

Hiciste bien. El rango, las matrices no cuadradas y los mínimos cuadrados **no van en esta pregunta** — la pregunta 1 habla de invertibilidad, y la invertibilidad es un concepto de matrices cuadradas. Meterlo aquí se leería como que no $\in \psi$ diste el alcance.

El arreglo: que sea bisagra, no aside

En vez de anunciarla, la usas para conectar los dos párrafos:

...y, al multiplicar ambos lados por Q^T , queda reducido a $Rx = Q^Tb$. Este primer paso no depende de que A sea invertible: la inversa de Q existe por el solo hecho de ser ortogonal. La invertibilidad de A interviene en el paso siguiente.

En efecto, como A invertible equivale a que sus columnas sean linealmente independientes...

Ahora ya no es información extra: es la **transición** que le da estructura a tu respuesta. Cierra el párrafo 1 y abre el 2.

Si prefieres quitarla

También es defendible. Quedaría así, ajustando el arranque del segundo párrafo para que no quede colgando:

...y, al multiplicar ambos lados por Q^T , queda reducido a $Rx = Q^Tb$.

Ahora bien, la invertibilidad de A garantiza además que R sea invertible. Como A invertible equivale a que sus columnas sean linealmente independientes, y cada elemento diagonal de R mide el aporte independiente de la columna correspondiente, ninguno de ellos se anula ($r_{ii} \neq 0$). En consecuencia el sistema triangular puede resolverse mediante sustitución hacia atrás...

Se lee bien y es más corto. Pierdes la precisión de *dónde* actúa la invertibilidad, que es lo único que estarías sacrificando.

Mi recomendación: la versión bisagra. Es la misma cantidad de texto que tenías y responde la pregunta con más filo. Pero si al leerla en voz alta sigue sonándote a pegote, quítala sin culpa — tu segunda versión también responde bien lo que se pregunta.

Aunque sabes, honestamente no entiendo por qué conviene dejar esta precisión: Conviene notar que este primer paso no impone ninguna condición sobre A : la inversa de Q existe siempre y se obtiene sin necesidad de cálculo alguno, simplemente transponiendo.

Tu instinto es correcto, y el problema es mío: **el contenido sirve, la forma no**. Lo escribí como "conviene notar que...", que es literalmente anunciar una nota al margen. Así suena a dato suelto que metí porque sí.

Por qué el contenido sí importa

La pregunta es *dónde* se necesita la invertibilidad. Tu respuesta tiene dos pasos:

paso 1: $Ax = b \rightarrow Rx = Q^Tb$ (usar Q^T)
paso 2: resolver $Rx = Q^Tb$ (sustitución hacia atrás)

Si no dices nada, el lector no sabe si la invertibilidad se necesita en los dos, en uno, o en cuál. Con esa frase, tu respuesta afirma algo concreto: **la invertibilidad no hace falta en el paso 1, solo en el paso 2**. Eso es precisión, no adorno.

Y de paso te da el contraste con LU y PLU en el documento del grupo: allá la invertibilidad se necesita desde el principio, para que la factorización exista. Aquí no.

El arreglo: que sea bisagra, no aside

Tengo una duda, es posible que que QR pueda aplicarse o exista incluso cuando alguna columna es combinación lineal de las otras... Es que no recuerdo si era QR o COMO TAL Gram-Schmidt

Tu duda apunta exactamente a la distinción correcta, y las dos cosas son distintas:

La FACTORIZACIÓN QR $\rightarrow$ existe siempre
el MÉTODO Gram-Schmidt $\rightarrow$ se rompe si hay columnas dependientes

Tu párrafo está bien escrito, entonces. Pero te explico por qué, porque es sutil.

Por qué la factorización sigue existiendo

Supón que a_2 es el doble de a_1 :

$$a_1 = (1, 1) \quad a_2 = (2, 2)$$

Haces el proceso: normalizas a_1 y obtienes q_1 . Luego le restas a_2 a a_1 , su sombra sobre q_1 ... y no queda **nada**. El residuo es el vector cero.

Ahí Gram-Schmidt se queda sin instrucciones, porque su regla es "divide el residuo entre su norma" y no puede dividirl entre cero.

Pero fíjate en lo que sí puedes hacer a mano: como q_1 ya no tiene que representar ninguna dirección de a_1 (a, no traía nada nuevo), puedes escoger cualquier vector unitario perpendicular a q_1 . Por ejemplo $q_2 = (1/\sqrt{2}, -1/\sqrt{2})$.

Y con eso la factorización cuadra perfectamente:

$$a_1 = \sqrt{2} \cdot q_1 \quad \checkmark \\ a_2 = 2\sqrt{2} \cdot q_1 + 0 \cdot q_2 \quad \checkmark$$

$$a_1 = \sqrt{2} \cdot q_1 \quad \checkmark \\ a_2 = 2\sqrt{2} \cdot q_1 + 0 \cdot q_2 \quad \checkmark$$

El 0 en la diagonal de R es precisamente el que dice "esta columna *no* necesitó a q_2 ". La igualdad $A = QR$ se cumple, aunque R ya no sea invertible.

Entonces, ¿dónde está la diferencia?

En que la factorización admite una elección libre y el método no.

Cuando el residuo da cero, hay infinitas q_i válidas — cualquiera perpendicular a q_1 sirve. Gram-Schmidt es una receta determinista: tiene una sola fórmula y no contempla ese caso. Así que el algoritmo se detiene, pero el objeto matemático que buscaba sí existe.

Una consecuencia de esa libertad: cuando hay columnas dependientes, **Q y R dejan de ser únicas**. Con columnas independientes sí lo son.

(Dato que refuerza el punto: otros métodos para obtener QR, como las transformaciones de Householder, no se rompen con columnas dependientes. Producen Q y R sin problema. Eso confirma que el obstáculo era del procedimiento, no de la factorización.)

Tu texto, entonces

Está correcto tal como lo tienes:

La factorización $A = QR$ existe siempre, incluso cuando alguna columna es combinación de las otras. Lo que la independencia lineal garantiza no es que la factorización exista, sino que el proceso de Gram-Schmidt pueda llevarse a cabo y que R resulte invertible.

Esa frase es justo la distinción que acabas de intuir, y es de las cosas que más van a destacar en tu respuesta — porque el error común es decir "QR requiere columnas independientes", que mezcla las dos cosas.

Si quieres blindarla del todo, podrías añadirle media línea al final: "...y que Q y R sean únicas". Pero no es necesario.

Errores detectados en las respuestas de la herramienta

Se incluyen a continuación las capturas de los casos en que la herramienta entregó respuestas incorrectas o imprecisas y fueron corregidas por el grupo. Se dejan como evidencia del proceso de verificación descrito en la sección anterior.

Modificación sobre el sistema original: consiste en multiplicar A por la derecha por una matriz de permutación P, obteniendo $AP = QR$. Nótese la diferencia con PLU: allí la permutación va por la izquierda y reordena filas; aquí va por la derecha y reordena columnas.

¿Cuándo es indispensable realizarla?

1. Por imposibilidad algebraica (columnas dependientes): cuando una columna es combinación de las anteriores, al restarle sus proyecciones no queda residuo alguno y el proceso no puede normalizarla, pues habría que dividir entre cero. Esto se refleja en un cero en la diagonal de R.

2. Por estabilidad numérica (columnas casi dependientes): aunque el residuo no sea exactamente cero, si es muy pequeño, dividir entre él amplifica los errores de redondeo, igual que ocurre con un pivote pequeño en la eliminación gaussiana.

Procedimiento de la modificación: en cada etapa se escoge, entre las columnas que faltan, aquella a la que le quede el residuo más grande, y se la coloca de siguiente mediante P. Así los valores de la diagonal de R quedan ordenados de mayor a menor y las columnas problemáticas se dejan para el final, donde su efecto queda aislado.

Por último, conviene distinguir entre **modificar la matriz y cambiar el algoritmo**: cuando el problema es solo la pérdida de precisión, no se altera A, sino que se reemplaza el Gram-Schmidt clásico por versiones numéricamente más estables (Gram-Schmidt modificado, o las transformaciones de Householder), que dan la misma factorización con mayor exactitud.

La única situación que justifica una modificación es que la condición misma falle: que alguna columna sea combinación de las otras, o que lo sea casi, en cuyo caso el residuo correspondiente resulta demasiado pequeño y dividir entre él amplifica los errores de redondeo. Para esos casos existe una variante llamada QR con pivoteo de columnas, en la que se reordenan las columnas de A colocando primero las que más aportan, y que se escribe $AP = QR$. Nótese que aquí la permutación reordena columnas y no filas como en PLU, de modo que lo que cambia es el orden de las incógnitas y basta con reordenar la solución al final.

Finalmente, conviene distinguir entre modificar la matriz y cambiar el algoritmo. Cuando el problema es solo la pérdida de precisión al trabajar con decimales, A no se altera: lo que se reemplaza es el Gram-Schmidt clásico por versiones numéricamente más estables, como el Gram-Schmidt modificado o las transformaciones de Householder, que producen la misma factorización con mayor exactitud.

Qué cambió

El titular pasó al principio. Ahora lo primero que se lee es "prácticamente nunca", que es la respuesta. Antes eso quedaba enterrado.

El pivoteo de columnas bajó de rango. Ya no es "la modificación de QR" en paralelo a P en PLU, sino lo que es en realidad: una variante para el caso en que la hipótesis falla. Se menciona porque la pregunta pide hablar de modificaciones, pero sin inflarlo.

Agregué una línea que no estaba: permutar columnas reordena las incógnitas, no las ecuaciones. Por eso al final hay que devolver el orden de x. Es una diferencia real con PLU, donde se permuta b y ya.

Se fue la lista numerada. Esos dos casos ya no necesitan viñetas propias — al ser la excepción y no la regla, caben en una oración.

¿Cuándo es necesario realizar una modificación y en qué consiste?

Entendemos aquí por modificación un cambio sobre la matriz original del sistema — como la permutación de filas que introduce PLU — y no un cambio en el algoritmo empleado.

Bajo esa lectura, QR no requiere ninguna modificación. Mientras que el método de Crout puede detenerse ante una matriz perfectamente invertible por el solo hecho de cómo están ordenadas sus filas, lo que obliga a introducir P en QR ese problema no existe: el procedimiento nunca divide entre elementos de A, sino entre las normas de los residuos que va obteniendo, de modo que ningún orden de filas ni de columnas puede provocar una división entre cero.

El único caso en que Gram-Schmidt no puede llevarse a cabo es cuando alguna columna de A es combinación de las otras. Pero conviene notar que ese obstáculo es de naturaleza distinta al de LU: allí el problema estaba en el orden y se resolvía reordenando; aquí el problema está en la matriz misma. La dependencia lineal entre columnas no depende del orden en que se escriban, de modo que ninguna permutación de filas ni de columnas la elimina. En otras palabras, cuando QR no puede aplicarse no es porque la matriz necesite una modificación, sino porque no cumple la condición requerida.

Por qué esta versión es mejor

El punto nuevo es el del último párrafo, y es el más fino que has tenido en toda la pregunta:

En LU el obstáculo es el orden, y por eso una permutación lo arregla.
En QR el obstáculo es una propiedad de la matriz, y por eso ninguna permutación lo arregla.

Eso explica, en una sola idea, por qué existe PLU y por qué no existe un "FQR" equivalente. No es que a QR le falte un remedio — es que su condición no es del tipo que se arregla reordenando. Y es completamente in-scope: sale de lo que ya vieron.

La asunción queda declarada, que era tu idea, y además te protege: si el profe esperaba algo más, la frase inicial muestra que fue una decisión de alcance y no un olvido.

Una salvedad, para que decidas con todo a la vista

Existe el riesgo pequeño de que el profe sí esperara que mencionaran el pivoteo de columnas. Si quieres cubrirte sin romper el tono, bastaría media línea al final:

Me parece que esta respuesta está incorrecta, porque Qué tiene que ver la matriz de permutación $AP = QR$, es decir, entiendo el contraste, pero se está hablando propiamente de QR, no de $AP = QR$...

Tienes razón, y es una buena captura. El contenido no era falso, pero el énfasis sí estaba mal — y en esta pregunta el énfasis es la respuesta.

Al dedicarle tres párrafos al pivoteo de columnas, mi versión hacía parecer que QR necesita remedios igual que LU. Y la respuesta verdadera es la contraria: para QR, prácticamente nunca hay que modificar nada. Eso es lo que había que decir de entrada, no al final y entre líneas.

Va corregido:

¿Cuándo es necesario realizar una modificación y en qué consiste?

En el caso de QR, prácticamente nunca, y esa es justamente la diferencia frente a LU. Mientras que el método de Crout puede detenerse ante una matriz perfectamente invertible, únicamente por el orden en que están dispuestas sus filas — lo que obliga a introducir la permutación P —, en QR eso no ocurre: si las columnas de A son independientes, la factorización se obtiene directamente, sin alterar la matriz.

Es que yo no digo que esto no sea cierto, sino que yo no tengo presente que hayamos visto estas modificaciones en clase del Grand-Schmidt modificado con una matriz de permutación... Yo creo que a lo mejor estás interpretando la pregunta muy a lo literal...

¿Bajo qué condiciones sobre la matriz puede aplicarse cada una directamente y cuándo es necesario realizar alguna modificación a la matriz original del sistema para poder aplicar cada una?

Porque yo creo que podemos asumir que no hablamos de modificaciones de un algoritmo en general, PODRÍAMOS ESO SÍ, dejar en claro esta asunción

La única situación que justifica una modificación es que la condición misma falle: que alguna columna sea combinación de las otras, o que lo sea casi, en cuyo caso el residuo correspondiente resulta demasiado pequeño y dividir entre él amplifica los errores de redondeo. Para esos casos existe una variante llamada QR con pivoteo de columnas, en la que se reordenan las columnas de A colocando primero las que más aportan, y que se escribe $AP = QR$. Nótese que aquí la permutación reordena columnas y no filas como en PLU, de modo que lo que cambia es el orden de las incógnitas y basta con reordenar la solución al final.

Finalmente, conviene distinguir entre modificar la matriz y cambiar el algoritmo. Cuando el problema es solo la pérdida de precisión al trabajar con decimales, A no se altera: lo que se reemplaza es el Gram-Schmidt clásico por versiones numéricamente más estables, como el Gram-Schmidt modificado o las transformaciones de Householder, que producen la misma factorización con mayor exactitud.

Mostrar menos

Proceso de pensamiento

Tu criterio es el correcto, y el argumento es bueno: si no lo vieron en clase, meterlo no suma y sí puede leerse como relleno. Mejor acotar el alcance y decirlo.

Y hay algo aún mejor: al quitar el pivoteo, aparece un punto más fuerte que el que estaba tapando.

Una salvedad, para que decidas con todo a la vista

Existe el riesgo pequeño de que el profe sí esperara que mencionaran el pivoteo de columnas. Si quieres cubrirte sin romper el tono, bastaría media línea al final:

Existen variantes de QR que reordenan columnas para tratar los casos de columnas casi dependientes, aunque exceden el alcance de lo visto en clase.

Una sola oración, sin desarrollar. Pero sí sabes que no se mencionó, yo la dejaría por fuera: tu versión responde completa y se lee más segura.

¿Y qué pasa si te pasas? Se da la vuelta, como el odómetro de un carro viejo.

```
python
a = np.int64(9223372036854775807) # el máximo
a + 1                             # → -9223372036854775808
```

Sumaste 1 y el número se volvió negativo. Sin error, sin aviso. Eso es exactamente lo que le pasó a tus datos.

float64 — rango enorme, pero solo 16 cifras

Guarda números con decimales. Llega hasta 1.8×10^{308} , muchísimo más lejos. Pero solo conserva unas 16 cifras significativas:

```
python
1e16 + 1 == 1e16 # → True (el 1 desapareció)
0.1 + 0.2        # → 0.30000000000000004
```

Y si te pasas del rango, no se da la vuelta: se vuelve **inf** (infinito).

El tercero, que casi nadie menciona

El **int** normal de Python no tiene límite:

```
python
2**200
# 1680938044258990275541962092341162602522202993782792835301376
```

Calcula con tantos dígitos como haga falta. Es más lento, pero es exacto siempre. Guárdate esto, porque es la clave de tu pregunta 3.

Tu algoritmo puede ser impecable y la respuesta seguir siendo basura. Si los datos de entrada están mal, ningún método lo arregla. *Garbage in, garbage out.*

Hay señales de alarma que se aprenden a reconocer: potencias altas de números grandes, columnas con magnitudes muy dispares, polinomios de grado alto sobre un intervalo ancho. Cuando las ves, el resultado va a ser malo antes de calcular nada.

Y lo más útil: qué se hace en la práctica

Nadie interpola con Vandermonde en grado alto. Las salidas reales son:

Escalar los datos. Si en vez de usar x de 0 a 38 los llevas al intervalo $[-1, 1]$, el condicionamiento mejora brutalmente. Lo medí:

```
cond con x en [0, 38] : 9.44 × 10^30
cond con x en [-1, 1] : 2.72 × 10^8
mejora: 3.5 × 10^22 veces
```

Sigue siendo malo, pero pasó de imposible a manejable. Y es un cambio de dos líneas de código.

Bajar el grado. Con 200 puntos, casi nunca quieres un polinomio de grado 199. Quieres uno de grado 3 que pase cerca de todos — y eso es mínimos cuadrados, justo donde QR brilla.

Usar splines (tramos de polinomios de grado bajo pegados) en vez de un polinomio gigante.

```
python
def valorExacto(x, n):
    suma = 0
    for k in range(n):
        suma += int(x) ** k # int de Python: sin limite
    return suma

for i in range(len(x20)):
    exacto = valorExacto(x20[i], 20)
    print(x20[i], int(y20[i]) == exacto)
```

Resultado: True en los primeros 5, False en los 15 restantes.

d) Verificar contra anclas conocidas

Si sabes que $p(0) = 1$ y $p(1) = 20$, compruébalo. Son puntos de control gratuitos.

La regla general

Cuando sospeches de un dato, recálculalo en un tipo más grande y compara. Si el resultado cambia, el original estaba mal.

Es el mismo principio que ya usas sin saberlo: cuando una cuenta te da raro, la rehaces de otra forma a ver si coincide.

Resumen

	Exactitud	Rango	Al pasarse
int64	exacta	$\pm 9.2 \times 10^{18}$	se da la vuelta (silencioso)
float64	~16 cifras	$\pm 1.8 \times 10^{308}$	se vuelve inf
int de Python	exacta	ilimitado	no se pasa

En tu ejercicio se cruzaron los dos problemas: los **y** desbordaron **int64** (se dieron la vuelta) y la matriz de Vandermonde de 200 desbordó **float64** (se volvió **inf**).

2. Cómo interpretarlo y para qué sirve en el mundo real

Qué representa el problema

La interpolación es ajustar una curva a unos datos. Eso está en todas partes: calibrar un sensor, reconstruir una señal, estimar una tendencia, rellenar mediciones faltantes.

El sistema $Va = y$ es la versión más directa de ese problema: "encuétrame el polinomio que pasa por estos puntos".

Las tres lecciones que deja

Un problema puede ser resoluble en matemáticas e imposible en el computador. El polinomio de grado 199 existe, es único, y está perfectamente definido. Y aun así no se puede calcular, porque las potencias no caben en la máquina. La existencia matemática no garantiza la computabilidad.

Tu algoritmo puede ser impecable y la respuesta seguir siendo basura. Si los datos de entrada están mal, ningún método lo arregla. *Garbage in, garbage out.*

3. Cómo comprobar si los datos están corrompidos

Cuatro técnicas, de la más rápida a la más definitiva.

a) Buscar violaciones de sentido común

Tus **y** deberían ser positivos y crecientes (son sumas de potencias de números positivos). En cuanto ves un negativo, ya sabes:

```
python
print("Valores negativos:", (y20 < 0).sum()) # → 8
```

Ocho negativos donde no debería haber ninguno. Bandera roja inmediata.

b) Comparar contra el límite del tipo

```
python
print(np.iinfo(np.int64).max) # 9223372036854775807
```

Si tus valores esperados se acercan a esa cifra, vas a desbordar.

c) Recalcular con aritmética exacta (la definitiva)

4. Qué librerías usar

Las que ya conoces

numpy — la base de todo. Arreglos, operaciones vectorizadas, y un módulo de álgebra lineal muy completo:

```
python
np.linalg.cond(V) # número de condición
np.linalg.solve(A, b) # resolver el sistema
np.linalg.qr(V) # factorización QR (con Householder)
np.linalg.norm(V) # norma
np.vander(x, N, increasing=True) # matriz de Vandermonde en una línea
```

Por dentro no ejecuta Python: llama a **LAPACK**, una biblioteca escrita en Fortran desde los años 90 y refinada durante décadas. Por eso es muchísimo más rápida y más robusta que un bucle a mano.

scipy.linalg — lo mismo pero más completo. Te da control fino:

```
python
from scipy.linalg import lu_factor, lu_solve, qr, solve_triangular
```



```
Hay varias cositas que no entiendo que me gustaría que me explicaras. Entre ellas:
1) Qué es eso de int64 y float64?
2) Cómo puedo interpretar esta información y por qué me serviría en el mundo real algo así o qué está representando?
3) Cómo puedo comprobar si está corrompido o no??
4) Y qué librerías me recomiendas en general, por qué y cómo funcionan?

import numpy as np

# =====
# Polinomio de orden 20
# =====

x20 = np.array([
    0, 2, 4, 6, 8, 10, 12, 14, 16, 18,
    20, 22, 24, 26, 28, 30, 32, 34, 36, 38
])

y20 = np.array([
    1,
    1048575,
    366503875925,
    731231688012595,
    164703072086692425,
    -7335632962598440505,
    -1964501723438287779,
    -1894033625802624709,
    1229782938247303441,
    -792061141976150257,
    -2052364549866899995,
    -9003277803673866045,
    -7168164137338222503,
    -6426821785559194665,
    4435783745579263469,
```